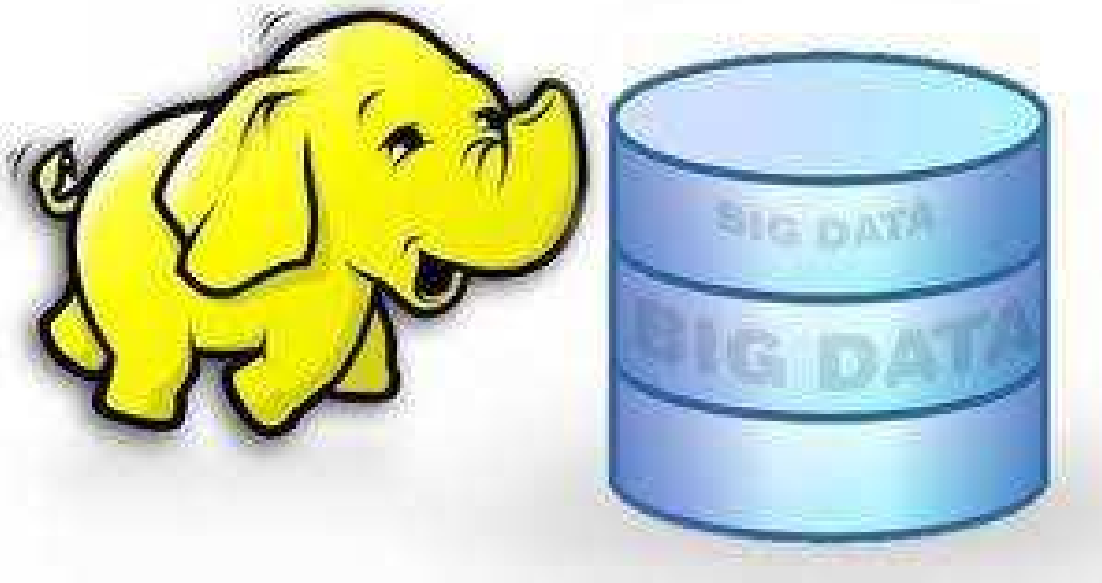


Chapitre 2

Hadoop



Hadoop: Présentation (1/3)

- **Hadoop** est une plateforme (framework) open source conçue pour réaliser d'une façon distribuée des traitements sur des volumes de données massives, de l'ordre de plusieurs pétaoctets. Ainsi, il est destiné à faciliter la création d'applications distribuées et échelonnables (scalables). Il s'inscrit donc typiquement sur le terrain du Big Data.
- **Hadoop** est géré sous l'égide de la fondation Apache, il est écrit en Java.
- **Hadoop** a été conçu par Doug Cutting en 2004 et été inspiré par les publications MapReduce, GoogleFS et BigTable de Google.

.

Hadoop: Présentation (2/3)

- Modèle simple pour les développeurs: il suffit de développer des tâches Map-Reduce, depuis des interfaces simples accessibles via des librairies dans des langages multiples (Java, Python, C/C++...).
- Déployable très facilement (paquets Linux pré-configurés), configuration très simple elle aussi.
- S'occupe de toutes les problématiques liées au calcul distribué, comme l'accès et le partage des données, la tolérance aux pannes, ou encore la répartition des tâches aux machines membres du cluster : le programmeur a simplement à s'occuper du développement logiciel pour l'exécution de la tâche.

Hadoop: Présentation (3/3)

Hadoop assure les critères des solutions Big Data:

- **Performance:** support du traitement d'énormes data sets (millions de fichiers, Go à To de données totales) en exploitant le parallélisme et la mémoire au sein des clusters de calcul.
- **Economie:** contrôle des coûts en utilisant de matériels de calcul de type standard.
- **Evolutivité (scalabilité):** un plus grand cluster devrait donner une meilleure performance.
- **Tolérance aux pannes:** la défaillance d'un nœud ne provoque pas l'échec de calcul.
- **Parallélisme de données:** le même calcul effectué sur toutes les données.

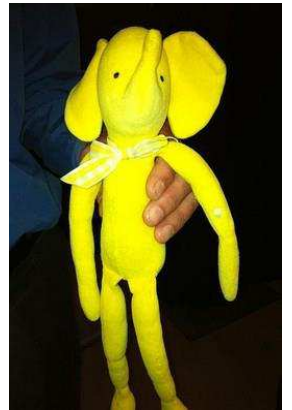
Hadoop : Historique (1/2)

- **2002:** Doug Cutting (directeur archive.org) et Mike Cafarella (étudiant) développent Nutch, un moteur de recherche Open Source exploitant le calcul distribué. L'implémentation peut tourner seulement sur quelques machines et a de multiples problèmes, notamment en ce qui concerne l'accès et le partage de fichiers.
- **2003/2004:** le département de recherche de Google publie deux whitepapers, le premier sur GFS (un système de fichier distribué) et le second sur le paradigme Map/Reduce pour le calcul distribué.
- **2004:** Doug Cutting et Mike Cafarella développent un framework (encore assez primitif) inspiré des papiers de Google et portent leur projet Nutch sur ce framework.
- **2006:** Doug Cutting (désormais chez Yahoo) est en charge d'améliorer l'indexation du moteur de recherche de Yahoo. Il exploite le framework réalisé précédemment...

Hadoop : Historique (2/2)

Il crée alors une nouvelle version améliorée du framework en tant que projet Open Source de la fondation Apache, qu'il nomme Hadoop (le nom d'un éléphant en peluche de son fils).

A l'époque, Hadoop est encore largement en développement – un cluster pouvait alors comporter au maximum 5 à 20 machines, etc.



- **2008:** le développement est maintenant très abouti, et Hadoop est exploité par le moteur de recherche de Yahoo Ainsi que par de nombreuses autres divisions de l'entreprise.
- **2011:** Hadoop est désormais utilisé par de nombreuses autres entreprises et des universités, et le cluster Yahoo comporte 42000 machines et des centaines de petaoctets d'espace de stockage.

Hadoop : Choix



Apache Hadoop



Outils & Support commercial avec :
outils graphiques pour le déploiement,
l'administration et le monitoring des clusters

Hadoop Distribution

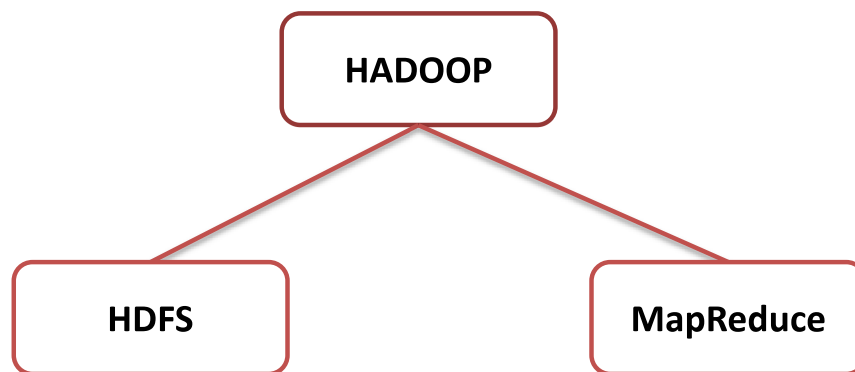


Plug ins / connecteurs additionnels
+
Modélisation & Configurateur graphique des jobs MR +
Planificateur

Big Data Suite

Hadoop: Composants fondamentaux

- Hadoop est constitué de deux grandes parties :
 - Hadoop Distributed File System – **HDFS** : destiné pour le stockage distribué des données
 - Distributed Programming Framework - **MapReduce** : destiné pour le traitement distribué des données.



HDFS : Présentation (1/3)

- **HDFS (Hadoop Distributed File System)** est un système de fichiers :
 - **Distribué**
 - **Extensible**
 - **Portable**
 - Développé par Hadoop, inspiré de GoogleFS et écrit en **Java**.
 - **Tolérant aux pannes**
 - Conçu pour stocker de **très gros volumes** de données sur un grand nombre de machines **peu coûteuses** équipées de disques durs banalisés.
 - Conçu pour les lectures en streaming, pas pour l'accès aléatoire
- **HDFS** permet l'**abstraction de l'architecture physique de stockage**, afin de manipuler un système de fichiers distribué comme s'il s'agissait d'un disque dur unique.
- **HDFS** reprend de nombreux concepts proposés par des systèmes de fichiers classiques comme **ext2 pour Linux ou FAT pour Windows**. Nous retrouvons donc la notion de **blocs** (la plus petite unité que l'unité de stockage peut gérer), les **métadonnées** qui permettent de retrouver les blocs à partir d'un nom de fichier, les **droits** ou encore l'**arborescence** des répertoires.

HDFS : Présentation (2/3)

- Toutefois, HDFS se démarque d'un système de fichiers classique pour les principales raisons suivantes :
 - HDFS n'est pas **solidaire du noyau du système d'exploitation**. Il assure une **portabilité** et peut être déployé sur différents systèmes d'exploitation. Un des inconvénients est de devoir solliciter une application externe pour monter une unité de disque HDFS.
 - HDFS est un système **distribué**. Sur un **système classique**, la **taille** du disque est généralement considérée comme la **limite globale** d'utilisation. Dans un système **distribué** comme HDFS, chaque **nœud** d'un cluster correspond à un **sous-ensemble** du volume global de données du cluster. **Pour augmenter ce volume global, il suffira d'ajouter de nouveaux nœuds**. On retrouvera également dans HDFS, un service central appelé Namenode qui aura la tâche de gérer les métadonnées.

HDFS : Présentation (3/3)

- HDFS utilise des **tailles de blocs largement supérieures** à ceux des systèmes classiques. Par défaut, la taille est fixée à **64 Mo**. Il est toutefois possible de monter à 128 Mo, 256 Mo, 512 Mo voire 1 Go. Alors que sur des systèmes classiques, la taille est généralement de **4 Ko**, l'intérêt de fournir des tailles plus grandes **permet de réduire le temps d'accès** à un bloc. Notez que si la taille du fichier est inférieure à la taille d'un bloc, le fichier n'occupera pas la taille totale de ce bloc.
- HDFS fournit un **système de réplication des blocs** dont le nombre de répliques est configurable. Pendant la phase d'écriture, chaque bloc correspondant au fichier est répliqué sur plusieurs nœuds. Pour la phase de lecture, si un bloc est indisponible sur un nœud, des copies de ce bloc seront disponibles sur d'autres nœuds.
- HDFS peut créer, supprimer, copier, mais PAS mettre à jour les données
- HDFS accès aux données via MapReduce

HDFS : Architecture (1/3)

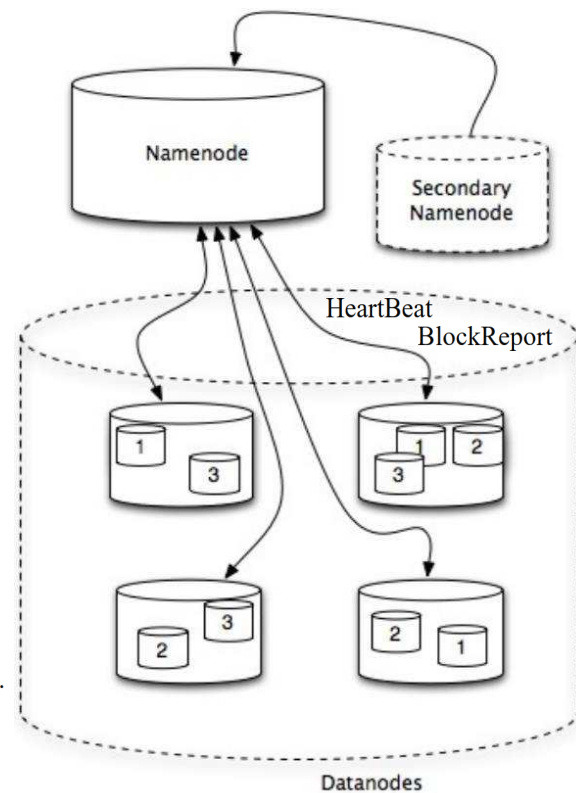
- Une architecture de machines HDFS (aussi appelée cluster HDFS) repose sur trois types de composants majeurs :

— NameNode (noeud de nom) :

Un Namenode est un **service central** (généralement appelé aussi **maître**) qui s'occupe de **gérer l'état** du système de fichiers. Il **maintient l'arborescence** du système de fichiers et **les métadonnées** de l'ensemble des fichiers et répertoires d'un système Hadoop. Le Namenode a une connaissance des Datanodes (étudiés juste après) dans lesquels les blocs sont stockés.

Le NameNode reçoit régulièrement des notifications de ses DataNodes :

- Un "**HeartBeat**", pour s'assurer du fonctionnement correct du DataNode
- Un "**BlockReport**", pour savoir la liste de blocs présents dans le DataNode émetteur.



Architecture Maître/Escalve

HDFS : Architecture (2/3)

– Secondary Namenode :

Le Namenode dans l'architecture Hadoop est un **point unique de défaillance** (Single Point of Failure en anglais). Si ce service est arrêté, il n'y a pas un moyen de pouvoir extraire les blocs d'un fichier donné. Pour répondre à cette problématique, un Namenode secondaire appelé Secondary Namenode a été mis en place dans l'architecture **Hadoop version2**. Son fonctionnement est relativement simple puisque le **Namenode secondaire vérifie périodiquement l'état du Namenode principal et copie les métadonnées**. Si le **Namenode principal est indisponible, le Namenode secondaire prend sa place**.

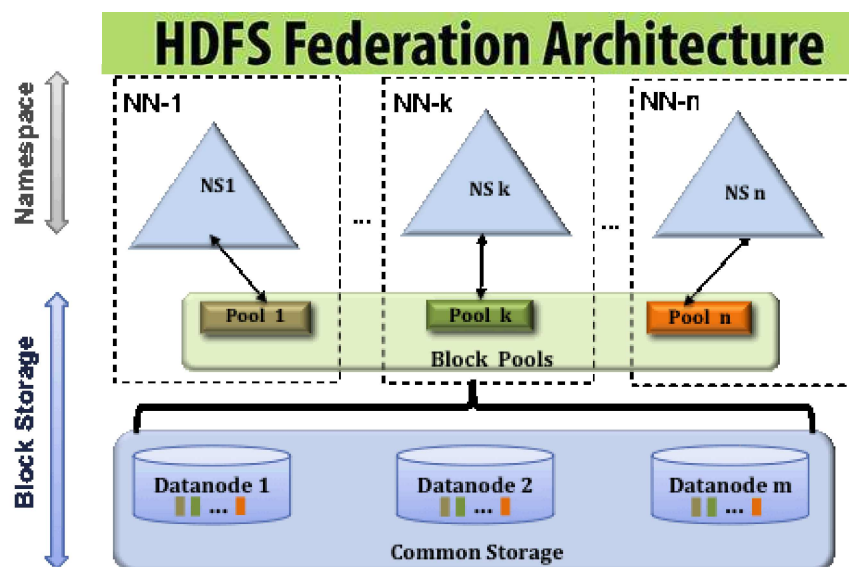
– Datanode :

Précédemment, nous avons vu qu'un Datanode **contient les blocs de données**. En effet, il stocke les blocs de données lui-mêmes. Il y a un DataNode pour chaque machine au sein du cluster. Les Datanodes sont sous les ordres du Namenode et sont surnommés les **Workers**. Ils sont donc sollicités par les Namenodes lors des opérations de lecture et d'écriture.

HDFS : Architecture (3/3)

- **Remarque : En plus de l'ajout du Namenode secondaire, nous trouvons d'autres améliorations de HDFS dans la 2^{ème} version de Hadoop :**

Les NameNodes étant le point unique pour le stockage et la gestion des métadonnées, ils peuvent être un goulot d'étranglement pour soutenir un grand nombre de fichiers, notamment lorsque ceux-ci sont de petite taille. **En acceptant des espaces de noms multiples desservis par des NameNodes séparés, le HDFS limite ce problème.** Par exemple, un NameNode pourrait gérer tous les fichiers sous /user, et un second NameNode peut gérer les fichiers sous /share.

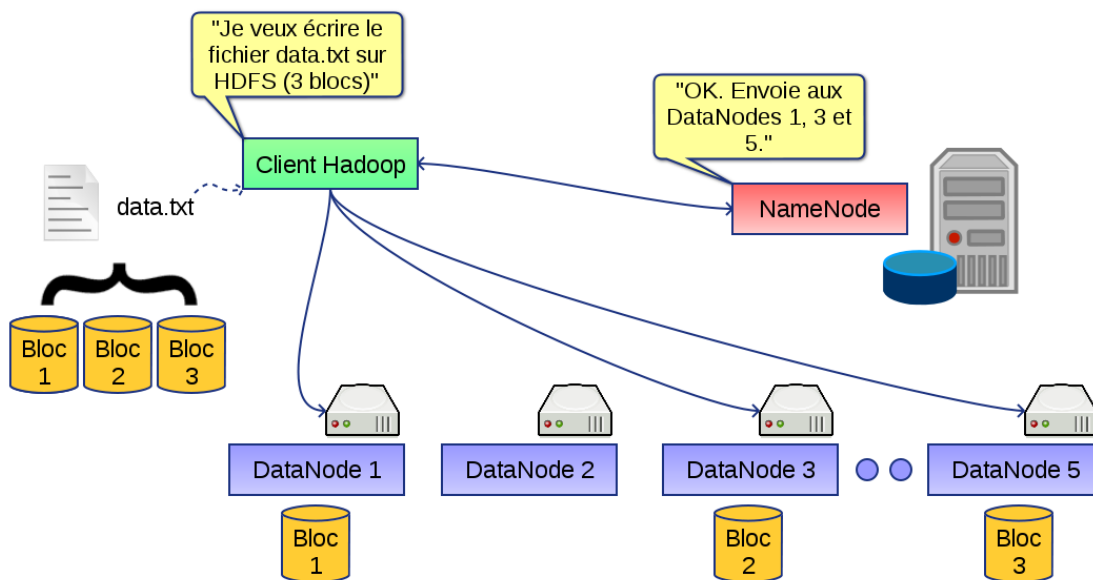


HDFS: Écriture d'un fichier (1/2)

- Si on souhaite écrire un fichier au sein de HDFS, on va utiliser la commande principale de gestion de Hadoop: **hadoop**, avec l'option **fs**. Mettons qu'on souhaite stocker le fichier `page_livre.txt` sur HDFS.
- Le programme va diviser le fichier en blocs de 64 Mo (ou autre, selon la configuration) – supposons qu'on ait ici 2 blocs. Il va ensuite annoncer au NameNode: « Je souhaite stocker ce fichier au sein de HDFS, sous le nom `page_livre.txt` ».
- Le NameNode va alors indiquer au programme qu'il doit stocker le bloc 1 sur le DataNode numéro 3, et le bloc 2 sur le DataNode numéro 1.
- Le client hadoop va alors contacter directement les DataNodes concernés et leur demander de stocker les deux blocs en question. Par ailleurs, les DataNodes s'occuperont – en informant le NameNode – de répliquer les données entre eux pour éviter toute perte de données.

HDFS: Écriture d'un fichier (2/2)

Ecriture HDFS



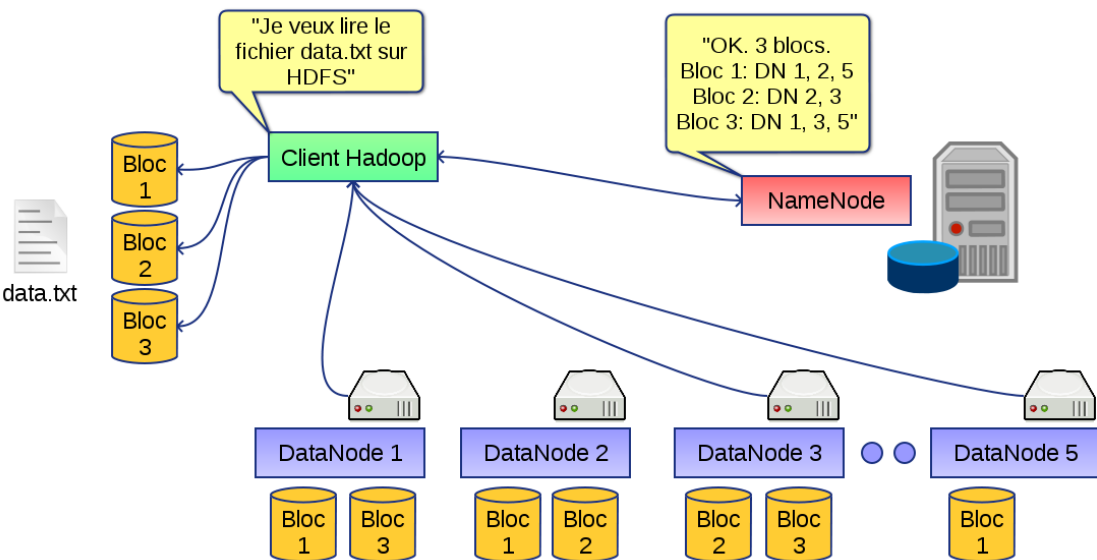
- Le client indique au NameNode qu'il souhaite écrire un bloc.
- Celui-ci lui indique le DataNode à contacter.
- Le client envoie le bloc au Datanode.
- Les DataNodes répliquent le bloc entre eux.
- Le cycle se répète pour le bloc suivant.

HDFS: Lecture d'un fichier (1/2)

- Si on souhaite lire un fichier au sein de HDFS, on utilise là aussi le client Hadoop. Mettons qu'on souhaite lire le fichier `page_livre.txt`.
- Le client va contacter le NameNode, et lui indiquer « Je souhaite lire le fichier `page_livre.txt` ». Le NameNode lui répondra par exemple « Il est composé de deux blocs. Le premier est disponible sur le DataNode 3 et 2, le second sur le DataNode 1 et 3 ».
- Là aussi, le programme contactera les DataNodes directement et leur demandera de lui transmettre les blocs concernés. En cas d'erreur/non réponse d'un des DataNode, il passe au suivant dans la liste fournie par le NameNode.

HDFS: Lecture d'un fichier (2/2)

Lecture HDFS



- Le client indique au NameNode qu'il souhaite lire un fichier.
- Celui-ci lui indique sa taille et les différents DataNode contenant les N blocs.
- Le client récupère chacun des blocs à un des DataNodes.
- Si un DataNode est indisponible, le client le demande à un autre.

HDFS: La commande Hadoop fs

- Comme indiqué plus haut, la commande permettant de stocker ou extraire des fichiers de HDFS est l'utilitaire console hadoop, avec l'option fs. Il réplique globalement les commandes systèmes standard Linux, et est très simple à utiliser:
- `hadoop fs -put livre.txt /data_input/livre.txt`
 - Pour stocker le fichier `livre.txt` sur HDFS dans le répertoire `/data_input`.
- `hadoop fs -get /data_input/livre.txt livre.txt`
 - Pour obtenir le fichier `/data_input/livre.txt` de HDFS et le stocker dans le fichier local `livre.txt`.
- `hadoop fs -mkdir /data_input`
 - Pour créer le répertoire `/data_input`
- `hadoop fs -rm /data_input/livre.txt`
 - Pour supprimer le fichier `/data_input/livre.txt`
- D'autres commandes usuelles: `-ls`, `-cp`, `-rmr`, `-du`, etc...

MapReduce : Présentation (1/4)

- Pour exécuter un **problème** large de **manière distribuée**, il faut pouvoir **découper** le problème en plusieurs problèmes de taille réduite à **exécuter sur chaque machine du cluster** (stratégie algorithmique dite du divide and conquer / diviser pour régner).
- De multiples approches existent et ont existé pour cette division d'un problème en plusieurs « sous-tâches ».
- MapReduce est un **paradigme** (un **modèle**) visant à généraliser les approches existantes pour produire une **approche unique** applicable à tous les problèmes.
- MapReduce existait déjà depuis longtemps, notamment dans les langages fonctionnels (Lisp, Scheme), mais la présentation du paradigme sous une forme « **rigoureuse** », **généralisable** à tous les problèmes et orientée **calcul distribué** est attribuable à un whitepaper issu du département de recherche de **Google** publié en **2004** (« MapReduce: Simplified Data Processing on Large Clusters »).

MapReduce : Présentation (2/4)

MapReduce définit deux opérations distinctes à effectuer sur les données d'entrée:

- **La première, MAP**, va transformer les données d'entrée en une série de **couples clef/valeur**. Elle va regrouper les données en les associant à des clefs, choisies de telle sorte que les couples clef/valeur aient un sens par rapport au problème à résoudre. Par ailleurs, cette opération doit être **parallélisable**: on doit pouvoir découper les données d'entrée en plusieurs fragments, et faire **exécuter l'opération MAP à chaque machine du cluster** sur un fragment distinct.
- **La seconde, REDUCE**, va appliquer un **traitement** à toutes les valeurs de chacune des clefs distinctes produite par l'opération MAP. Au terme de l'opération REDUCE, on aura **un résultat pour chacune des clefs distinctes**. Ici, on attribuera à chacune des machines du cluster une des clefs uniques produites par MAP, en lui donnant la liste des valeurs associées à la clef. **Chacune des machines effectuera alors l'opération REDUCE** pour cette clef.

MapReduce : Présentation (3/4)

On distingue donc 4 étapes distinctes dans un traitement MapReduce:

- **Découper** (split) les données d'entrée en plusieurs fragments.
- **Mapper** chacun de ces fragments pour obtenir des couples (clef ; valeur).
- **Grouper** (shuffle) ces couples (clef ; valeur) par clef.
- **Réduire** (reduce) les groupes indexés par clef en une forme finale, avec une valeur pour chacune des clefs distinctes.

En modélisant le problème à résoudre de la sorte, on le rend parallélisable – chacune de ces tâches à l'exception de la première seront effectuées de manière distribuée.

MapReduce : Présentation (4/4)

Pour résoudre un problème via la méthodologie MapReduce avec Hadoop, on devra donc:

- **Choisir une manière de découper les données d'entrée de telle sorte que l'opération MAP soit parallélisable.**
- **Définir** quelle **CLEF** utiliser pour notre problème.
- Écrire le **programme** pour l'opération **MAP**.
- Écrire le **programme** pour l'opération **REDUCE**..

... et Hadoop se chargera du reste (problématiques calcul distribué, groupement par clef distincte entre MAP et REDUCE, etc.).

MapReduce : Exemple concret (1/8)

- Imaginons qu'on nous donne un texte écrit en langue Française. On souhaite déterminer pour un travail de recherche quels sont les mots les plus utilisés au sein de ce texte (exemple Hadoop très répandu).
- Ici, nos données d'entrée sont constituées du contenu du texte.
- Première étape: déterminer une manière de découper (split) les données d'entrée pour que chacune des machines puisse travailler sur une partie du texte.
- Notre problème est ici très simple – on peut par exemple décider de découper les données d'entrée ligne par ligne. Chacune des lignes du texte sera un fragment de nos données d'entrée.

MapReduce : Exemple concret (2/8)

- **Nos données d'entrée (le texte):**

Celui qui croyait au ciel
Celui qui n'y croyait pas
[...]
Fou qui fait le délicat
Fou qui songe à ses querelles

(Louis Aragon, *La rose et le
Réséda*, 1943, fragment)

- **Pour simplifier les choses, on va avant le découpage supprimer toute ponctuation et tous les caractères accentués. On va également passer l'intégralité du texte en minuscules.**

MapReduce : Exemple concret (3/8)

- **Nos données d'entrée (le texte):**

celui qui croyait au ciel

celui qui ny croyait pas

fou qui fait le delicat

fou qui songe a ses querelles

- **... on obtient 4 fragments depuis nos données d'entrée.**

MapReduce : Exemple concret (4/8)

- On doit désormais déterminer la clef à utiliser pour notre opération MAP, et écrire le code de l'opération MAP elle-même.
- Puisqu'on s'intéresse aux occurrences des mots dans le texte, et qu'à terme on aura après l'opération REDUCE un résultat pour chacune des clefs distinctes, la clef qui s'impose logiquement dans notre cas est: le mot-lui même.
- Quand à notre opération MAP, elle sera elle aussi très simple: on va simplement parcourir le fragment qui nous est fourni et, pour chacun des mots, générer le couple clef/valeur: (MOT ; 1). La valeur indique ici l'occurrence pour cette clef - puisqu'on a croisé le mot une fois, on donne la valeur « 1 ».

MapReduce : Exemple concret (5/8)

- Le code de notre opération MAP sera donc (ici en pseudo code):

```
POUR MOT dans LIGNE, FAIRE :  
    GENERER COUPLE (MOT; 1)
```

- Pour chacun de nos fragments, les couples (clef; valeur) générés seront donc:

celui qui croyait au ciel	➡	(celui;1) (qui;1) (croyait;1) (au;1) (ciel;1)
celui qui ny croyait pas	➡	(celui;1) (qui;1) (ny;1) (croyait;1) (pas;1)
fou qui fait le delicat	➡	(fou;1) (qui;1) (fait;1) (le;1) (delicat;1)
fou qui songe a ses querelles	➡	(fou;1) (qui;1) (songe;1) (a;1) (ses;1) (querelles;1)

MapReduce : Exemple concret (6/8)

- Une fois notre opération MAP effectuée (de manière distribuée), Hadoop groupera (shuffle) tous les couples par clef commune.
- Cette opération est effectuée automatiquement par Hadoop. Elle est, là aussi, effectuée de manière distribuée en utilisant un algorithme de tri distribué, de manière récursive. Après son exécution, on obtiendra les 15 groupes suivants:

(celui;1) (celui;1)

(qui;1) (qui;1) (qui;1) (qui;1)

(croyait;1) (croyait;1)

(au;1) (ciel;1) (ny;1) (pas;1)

(fou;1) (fou;1) (fait;1) (le;1)

(delicat;1) (songe;1)

(a;1) (ses;1)

(querelles;1)

MapReduce : Exemple concret (7/8)

- Il nous reste à créer notre opération **REDUCE**, qui sera appelée pour chacun des groupes/clef distincte.
- Dans notre cas, elle va simplement consister à additionner toutes les valeurs liées à la clef spécifiée:

```
TOTAL=0  
POUR COUPLE dans GROUPE, FAIRE:  
    TOTAL=TOTAL+1  
RENVoyer TOTAL
```

MapReduce : Exemple concret (8/8)

- Une fois l'opération REDUCE effectuée, on obtiendra donc une valeur unique pour chaque clef distincte. En l'occurrence, notre résultat sera:

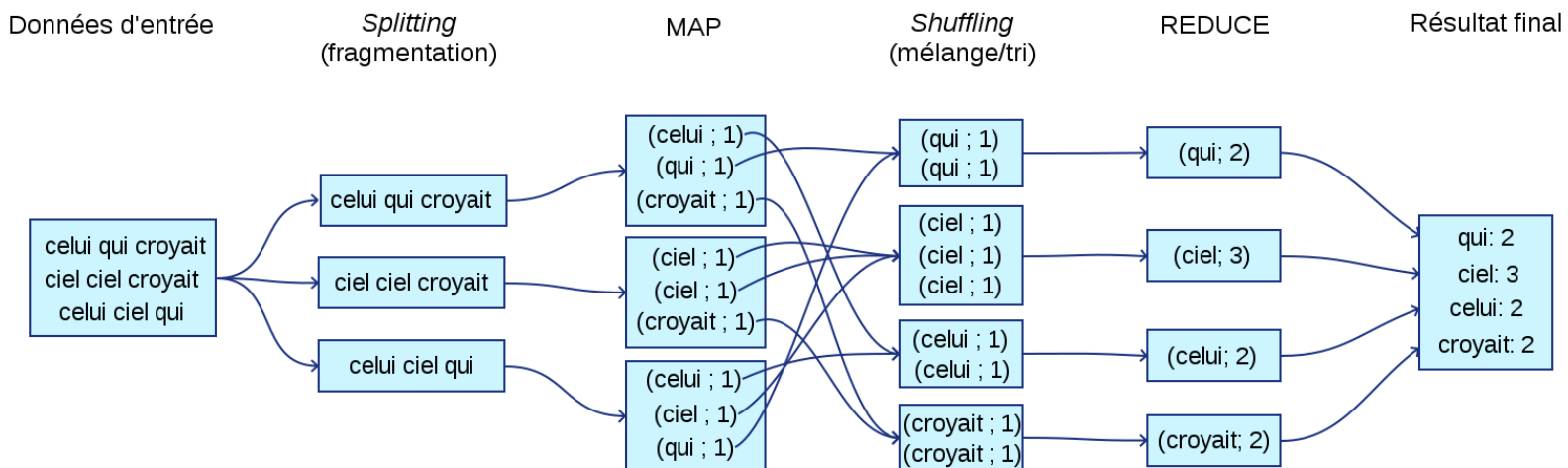
```
qui : 4  
celui : 2  
croyait : 2  
fou : 2  
au : 1  
ciel : 1  
ny : 1  
pas : 1  
fait : 1  
[...]
```

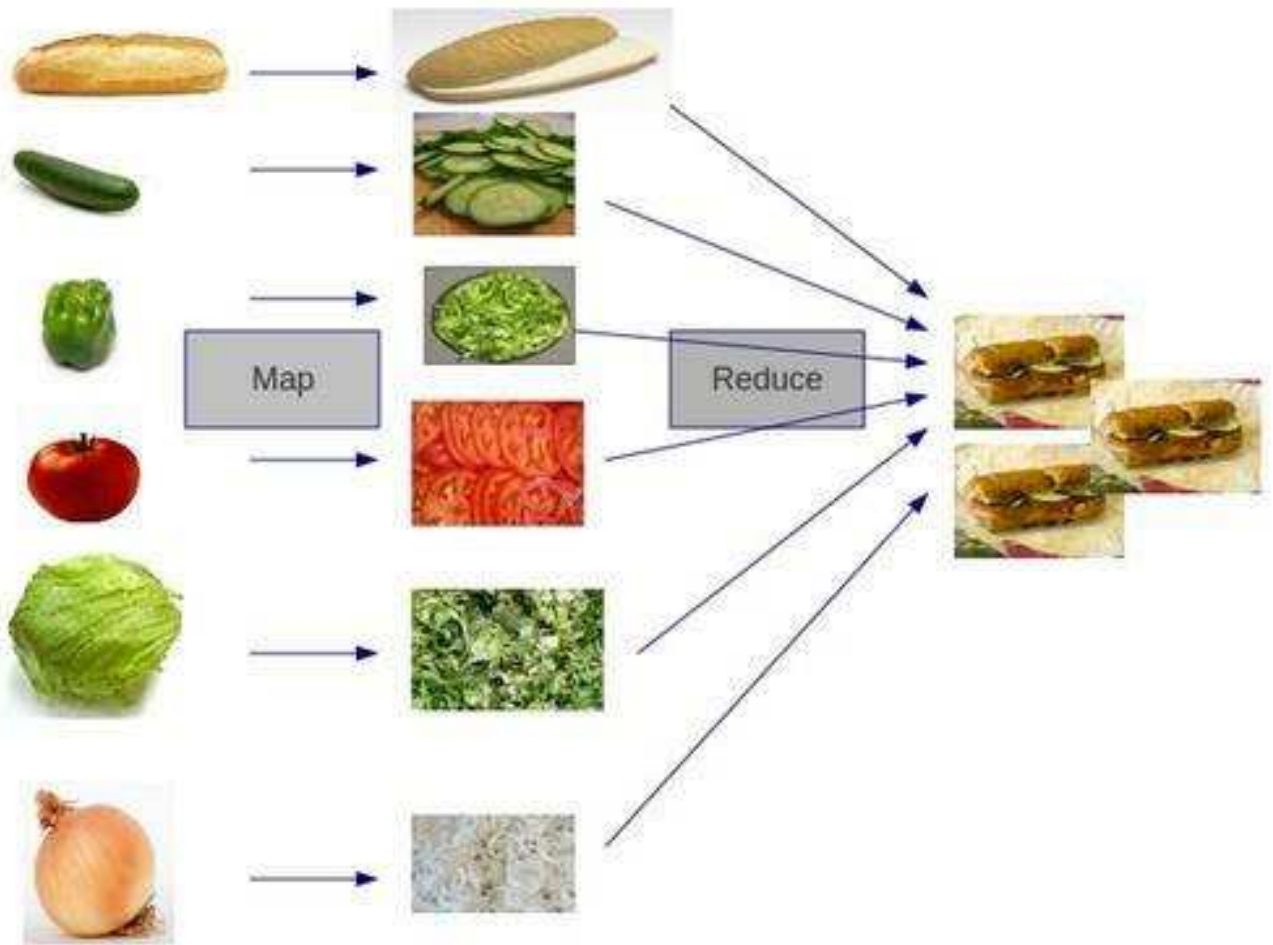
- On constate que le mot le plus utilisé dans notre texte est « qui », avec 4 occurrences, suivi de « celui », « croyait » et « fou », avec 2 occurrences chacun.

MapReduce : Exemple concret - Conclusion

- **Notre exemple est évidemment trivial, et son exécution aurait été instantanée même sur une machine unique, mais il est d'ores et déjà utile: on pourrait tout à fait utiliser les mêmes implémentations de MAP et REDUCE sur l'intégralité des textes d'une bibliothèque Française, et obtenir ainsi un bon échantillon des mots les plus utilisés dans la langue Française.**
- **L'intérêt du modèle MapReduce est qu'il nous suffit de développer les deux opérations réellement importantes du traitement: MAP et REDUCE, et de bénéficier automatiquement de la possibilité d'effectuer le traitement sur un nombre variable de machines de manière distribuée.**

MapReduce : Schéma général





Posté par Mirko Krivanek : « What Is MapReduce? », credit @Tgrall
<http://www.datasciencecentral.com/forum/topics/what-is-map-reduce>

MapReduce : Exemple – Statistiques web

- **Un autre exemple: on souhaite compter le nombre de visiteurs sur chacune des pages d'un site Internet. On dispose des fichiers de logs sous la forme suivante:**

```
/index.html [19/05/2017:18:45:03 +0200]  
/contact.html [19/05/2017:18:46:15 +0200]  
/news.php?id=5 [24/05/2017:18:13:02 +0200]  
/news.php?id=4 [24/05/2017:18:13:12 +0200]  
/news.php?id=18 [24/05/2017:18:14:31 +0200]  
...etc...
```

- **Ici, notre clef sera par exemple l'URL d'accès à la page, et nos opérations MAP et REDUCE seront exactement les mêmes que celles qui viennent d'être présentées: on obtiendra ainsi le nombre de vue pour chaque page distincte du site.**

MapReduce : Exemple – Graphe social (1/8)

- **Un autre exemple: on administre un réseau social comportant des millions d'utilisateurs.**
- **Pour chaque utilisateur, on a dans notre base de données la liste des utilisateurs qui sont ses amis sur le réseau (via une requête SQL).**
- **On souhaite afficher quand un utilisateur va sur la page d'un autre utilisateur une indication « Vous avez N amis en commun ».**
- **On ne peut pas se permettre d'effectuer une série de requêtes SQL à chaque fois que la page est accédée (trop lourd en traitement). On va donc développer des programmes MAP et REDUCE pour cette opération et exécuter le traitement toutes les nuits sur notre base de données, en stockant le résultat dans une nouvelle table.**

MapReduce : Exemple – Graphe social (2/8)

- Ici, nos données d'entrée sous la forme Utilisateur => Amis:

```
A => B, C, D  
B => A, C, D, E  
C => A, B, D, E  
D => A, B, C, E  
E => B, C, D
```

- Puisqu'on est intéressé par l'information « amis en commun entre deux utilisateurs » et qu'on aura à terme une valeur par clef, on va choisir pour clef la concaténation entre deux utilisateurs. Par exemple, la clef « A-B » désignera « les amis en communs des utilisateurs A et B ».
- On peut segmenter les données d'entrée là aussi par ligne.

MapReduce : Exemple – Graphe social (3/8)

- Notre opération MAP va se contenter de prendre la liste des amis fournie en entrée, et va générer toutes les clefs distinctes possibles à partir de cette liste. La valeur sera simplement la liste d'amis, telle quelle.
- On fait également en sorte que la clef soit toujours triée par ordre alphabétique (clef « B-A » sera exprimée sous la forme « A-B »).
- Ce traitement peut paraître contre-intuitif, mais il va à terme nous permettre d'obtenir, pour chaque clef distincte, deux couples (clef;valeur): les deux listes d'amis de chacun des utilisateurs qui composent la clef.

MapReduce : Exemple – Graphe social (4/8)

- Le pseudo code de notre opération MAP:

```
UTILISATEUR = [PREMIERE PARTIE DE LA LIGNE]
POUR AMI dans [RESTE DE LA LIGNE], FAIRE:
  SI UTILISATEUR < AMI:
    CLEF = UTILISATEUR+"-"+AMI
  SINON:
    CLEF = AMI+"-"+UTILISATEUR
  GENERER COUPLE (CLEF; [RESTE DE LA LIGNE])
```

- Par exemple, pour la première ligne: A => B, C, D

- On obtiendra les couples (clef;valeur):

("A-B"; "B C D")

("A-C"; "B C D")

("A-D"; "B C D")

MapReduce : Exemple – Graphe social (5/8)

- Pour la seconde ligne : `B => A, C, D, E`

➤ On obtiendra ainsi :

`("A-B"; "A C D E")`

`("B-C"; "A C D E")`

`("B-D"; "A C D E")`

`("B-E"; " A C D E")`

- Pour la troisième ligne : `C => A, B, D, E`

➤ On aura :

`("A-C"; "A B D E")`

`("B-C"; "A B D E")`

`("C-D"; "A B D E")`

`("C-E"; " A B D E")`

- ...et ainsi de suite pour nos 5 lignes d'entrée.

MapReduce : Exemple – Graphe social (6/8)

- Une fois l'opération MAP effectuée, Hadoop va récupérer les couples (clef;valeur) de tous les fragments et les grouper par clef distincte. Le résultat sur la base de nos données d'entrée :

Pour la clef "A-B " : valeurs "A C D E" et "B C D"
Pour la clef "A-C " : valeurs "A B D E" et "B C D"
Pour la clef "A-D " : valeurs "A B C E" et "B C D"
Pour la clef "B-C " : valeurs "A B D E" et "A C D E"
Pour la clef "B-D " : valeurs "A B C E" et "A C D E"
Pour la clef "B-E " : valeurs "A C D E" et "B C D"
Pour la clef "C-D " : valeurs "A B C E" et "A B D E"
Pour la clef "C-E " : valeurs "A B D E" et "B C D"
Pour la clef "D-E " : valeurs "A B C E" et "B C D"

- ... on obtient bien, pour chaque clef « USER1-USER2 », deux listes d'amis: les amis de USER1 et ceux de USER2.

MapReduce : Exemple – Graphe social (7/8)

- Il nous faut enfin écrire notre programme **REDUCE**. Il va recevoir en entrée toutes les valeurs associées à une clef. Son rôle va être très simple: déterminer quels sont les amis qui apparaissent dans les listes (les valeurs) qui nous sont fournies. Pseudo-code:

```
LISTE_AMIS_COMMUNS=[] // Liste vide au départ.  
SI LONGUEUR(VALEURS)!=2, ALORS: // Ne devrait pas se produire.  
    RENOYER ERREUR  
SINON :  
    POUR AMI DANS VALEURS[0], FAIRE :  
        SI AMI EGALEMENT PRESENT DANS VALEURS[1], ALORS :  
            // Présent dans les deux listes d'amis, on l'ajoute.  
            LISTE_AMIS_COMMUNS+=AMI  
RENOYER LISTE_AMIS_COMMUNS
```

MapReduce : Exemple – Graphe social (8/8)

- Après exécution de l'opération REDUCE pour les valeurs de chaque clef unique, on obtiendra donc, pour une clef « A-B », les utilisateurs qui apparaissent dans la liste des amis de A et dans la liste des amis de B. Autrement dit, on obtiendra la liste des amis en commun des utilisateurs A et B. Le résultat:

```
"A-B " : "C, D"  
"A-C " : "B, D"  
"A-D " : "B, C"  
"B-C " : "A, D, E"  
"B-D " : "A, C, E"  
"B-E " : "C, D"  
"C-D " : "A, B, E"  
"C-E " : "B, D"  
"D-E " : "B, C"
```

- On sait ainsi que A et B ont pour amis communs les utilisateurs C et D, ou encore que B et C ont pour amis communs les utilisateurs A, D et E.

MapReduce : Conclusion

- **En utilisant le modèle MapReduce, on a ainsi pu créer deux programmes très simples (nos programmes MAP et REDUCE) de quelques lignes de code seulement, qui permettent d'effectuer un traitement somme toute assez complexe.**
- **Mieux encore, notre traitement est parallélisable: même avec des dizaines de millions d'utilisateurs, du moment qu'on a assez de machines au sein du cluster Hadoop, le traitement sera effectué rapidement. Pour aller plus vite, il nous suffit de rajouter plus de machines.**
- **Pour notre réseau social, il suffira d'effectuer ce traitement toutes les nuits à heure fixe, et de stocker les résultats dans une table. Ainsi, lorsqu'un utilisateur visitera la page d'un autre utilisateur, un seul SELECT dans la base de données suffira pour obtenir la liste des amis en commun – avec un poids en traitement très faible pour le serveur.**

Architecture Hadoop: Présentation (1/3)

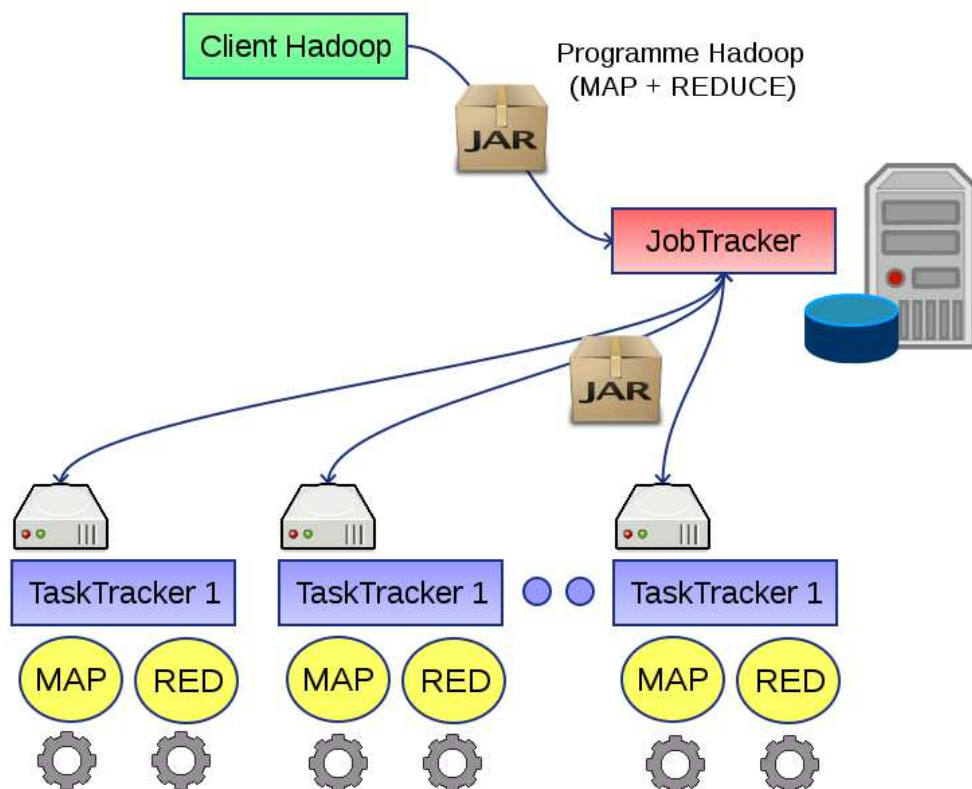
Comme pour HDFS, la gestion des tâches de Hadoop se base sur deux serveurs (des daemons):

- **Le JobTracker**, qui va directement recevoir la tâche à exécuter (un .jar Java), ainsi que les **données d'entrées** (nom des fichiers stockés sur HDFS) et le **répertoire où stocker les données de sortie** (toujours sur HDFS). Il y a un **seul** JobTracker sur une seule machine du cluster Hadoop. **Le JobTracker est en communication avec le NameNode de HDFS et sait donc où sont les données.**
- **Le TaskTracker**, qui est en communication constante avec le JobTracker et va recevoir les **opérations simples à effectuer (MAP/REDUCE)** ainsi que les **blocs de données correspondants** (stockés sur HDFS). Il y a un **TaskTracker sur chaque machine du cluster.**

Architecture Hadoop: Présentation (2/3)

- **Comme le JobTracker est conscient de la position des données (grâce au NameNode), il peut facilement déterminer les meilleures machines auxquelles attribuer les sous-tâches (celles où les blocs de données correspondants sont stockés).**
- **Pour effectuer un traitement Hadoop, on va donc stocker nos données d'entrée sur HDFS, créer un répertoire où Hadoop stockera les résultats sur HDFS, et compiler nos programmes MAP et REDUCE au sein d'un .jar Java.**
- **On soumettra alors le nom des fichiers d'entrée, le nom du répertoire des résultats, et le .jar lui-même au JobTracker: il s'occupera du reste (et notamment de transmettre les programmes MAP et REDUCE aux serveurs TaskTracker des machines du cluster).**

Architecture Hadoop: Présentation (3/3)



Architecture Hadoop: Le JobTracker (1/3)

Le déroulement de l'exécution d'une tâche Hadoop suit les étapes suivantes du point de vue du JobTracker :

1. Le client (un outil Hadoop console) va soumettre le travail à effectuer au JobTracker : une archive java .jar implémentant les opérations Map et Reduce. Il va également soumettre le nom des fichiers d'entrée et l'endroit où stocker les résultats.
2. Le JobTracker communique avec le NameNode HDFS pour savoir où se trouvent les blocs correspondant aux noms de fichiers donnés par le client.
3. Le JobTracker, à partir de ces informations, détermine quels sont les noeuds TaskTracker les plus appropriés, c'est à dire ceux qui contiennent les données sur lesquelles travailler sur la même machine, ou le plus proche possible (même rack/rack proche).
4. Pour chaque « morceau » des données d'entrée, le JobTracker envoie au TaskTracker sélectionné le travail à effectuer (MAP/REDUCE, code Java) et les blocs de données correspondants.

Architecture Hadoop: Le JobTracker (2/3)

5. Pour chaque « morceau » des données d'entrée, le JobTracker envoie au TaskTracker sélectionné le travail à effectuer (MAP/REDUCE, code Java) et les blocs de données correspondants.
6. Le JobTracker communique avec les noeuds TaskTracker en train d'exécuter les tâches. Ils envoient régulièrement un « **heartbeat** », un message signalant qu'ils travaillent toujours sur la sous-tâche reçue. Si aucun heartbeat n'est reçu dans une période donnée, le JobTracker considère la tâche comme ayant échoué et donne le même travail à effectuer à un autre TaskTracker.
7. Si par hasard une tâche échoue (erreur java, données incorrectes, etc.), le TaskTracker va signaler au JobTracker que la tâche n'a pas pu être exécutée.

Le JobTracker va alors décider de la conduite à adopter :

- Demander au même TaskTracker de ré-essayer.
- Redonner la sous-tâche à un autre TaskTracker.
- Marquer les données concernées comme invalides, etc.

Il pourra même **blacklister** le TaskTracker concerné comme **non-fiable** dans certains cas.

Architecture Hadoop: Le JobTracker (3/3)

7. Une fois que toutes les opérations envoyées aux TaskTracker (MAP + REDUCE) ont été effectuées et confirmées comme effectuées par tous les noeuds, le JobTracker marque la tâche comme « effectuée ». Des informations détaillées sont disponibles (statistiques, TaskTracker ayant posé problème, etc.).

Remarques

- Par ailleurs, on peut également obtenir à tout moment de la part du JobTracker des informations sur les tâches en train d'être effectuées: étape actuelle (MAP, SHUFFLE, REDUCE), pourcentage de complétion, etc.
- La soumission du .jar, l'obtention de ces informations, et d'une manière générale toutes les opérations liées à Hadoop s'effectuent avec le même unique client console vu précédemment: **hadoop** (avec d'autres options que l'option **fs** vu précédemment).

Architecture Hadoop: Le TaskTracker

- **Le TaskTracker dispose d'un nombre de « slots » d'exécution. A chaque « slot » correspond une tâche exécutable** (configurable). Ainsi, une machine ayant par exemple un processeur à 8 cœurs pourrait avoir 16 slots d'opérations configurées.
- Lorsqu'il reçoit une nouvelle tâche à effectuer (MAP, REDUCE, SHUFFLE) depuis le JobTracker, le **TaskTracker va démarrer une nouvelle instance de Java avec le fichier .jar** fourni par le JobTracker, **en appelant l'opération correspondante**.
- Une fois la tâche démarrée, il enverra **régulièrement** au JobTracker ses **messages heartbeats**. En dehors d'informer le JobTracker qu'il est toujours fonctionnels, ces messages indiquent également le nombre de slots disponibles sur le TaskTracker concerné.
- Lorsqu'une sous-tâche est terminée, le TaskTracker envoie un message au JobTracker pour l'en informer, que la tâche **est** bien déroulée ou non

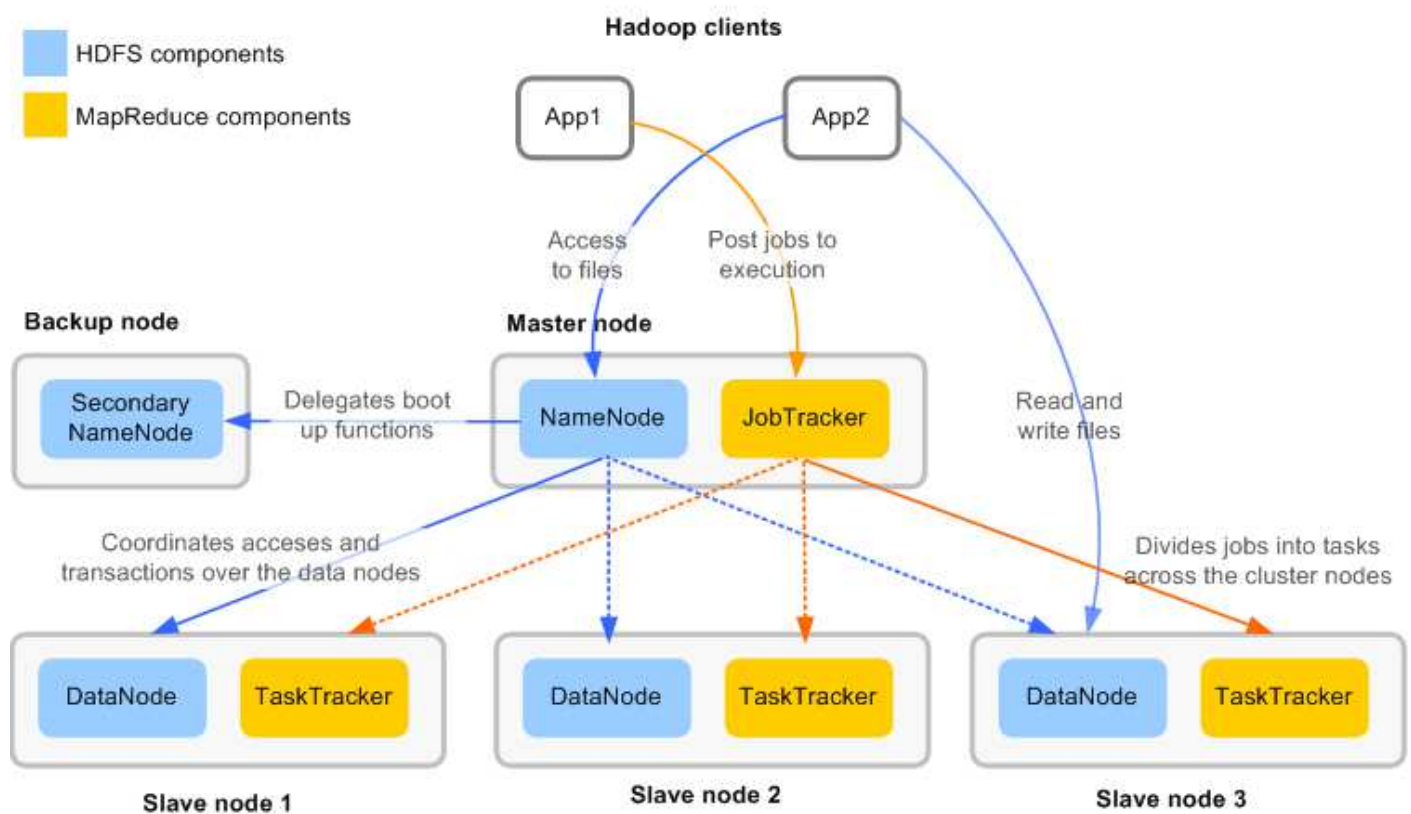
Architecture Hadoop: Remarques (1/2)

- De manière similaire au NameNode de HDFS, **il n'y a qu'un seul JobTracker** et **s'il tombe en panne, le cluster tout entier ne peut plus effectuer de tâches**. Là aussi, des résolutions aux problèmes sont ajoutées dans la version 2 de Hadoop (explication dans la section suivante).
- **Généralement, on place le JobTracker et le NameNode HDFS sur la même machine (une machine plus puissante que les autres), sans y placer de TaskTracker/DataNode HDFS pour limiter la charge.** Cette machine particulière au sein du cluster (qui contient les deux « gestionnaires », de tâches et de fichiers) est communément appelée le **noeud maître** (« **Master Node** »). Les autres noeuds (contenant TaskTracker + DataNode) sont communément appelés **noeuds esclaves** (« **slave node** »).

Architecture Hadoop: Remarques (2/2)

- Même si le JobTracker est situé sur une seule machine, le « client » qui envoie la tâche au JobTracker **initialement peut être exécuté sur n'importe quelle machine du cluster – comme les TaskTracker sont présents sur la machine, ils indiquent au client comment joindre le JobTracker.**
- La même remarque est valable pour l'accès au système de fichiers: les **DataNodes indiquent au client comment accéder au NameNode.**
- Enfin, tout changement de configuration Hadoop peut s'effectuer facilement simplement en changeant la configuration sur la machine où sont situés les serveurs NameNode et JobTracker: ils répliquent les changements de configuration sur tout le cluster automatiquement.

Architecture Hadoop: Architecture générale



YARN (MapReduce 2) : Présentation

- YARN (Yet-Another-Resource-Negotiator) est aussi appelé MRv2 (MapReduce 2). Ce n'est pas une refonte mais une évolution du framework MapReduce.
- YARN répond aux problématiques suivantes du Map Reduce :
 - Problème de limite de “Scalability” notamment par une meilleure séparation de la gestion de l'état du cluster et des ressources.
 - ~ 4000 Noeuds, 40 000 Tâches concourantes.
 - Problème d'allocation des ressources.

Cluster Summary (Heap Size is 12.58 GB/23.97 GB)

Running Map Tasks	Running Reduce Tasks	Total Submissions	Nodes	Occupied Map Slots	Occupied Reduce Slots	Reserved Map Slots	Reserved Reduce Slots	Map Task Capacity	Reduce Task Capacity	Avg. Tasks/Node
4512	285	2223	188	4512	285	0	0	4512	3008	40.00

Des tâches Map sont en attentes alors que des slots de tâche reduce sont libres.

Valeurs codées en dur.

YARN (MapReduce 2) : Architecture (1/7)

Le JobTracker a trop de responsabilités.

- Gérer les ressources du cluster.
- Gérer tous les jobs
 - Allouer les tâches et les ordonnancer.
 - Monitorer l'exécution des tâches.
 - Gérer le fail-over.*

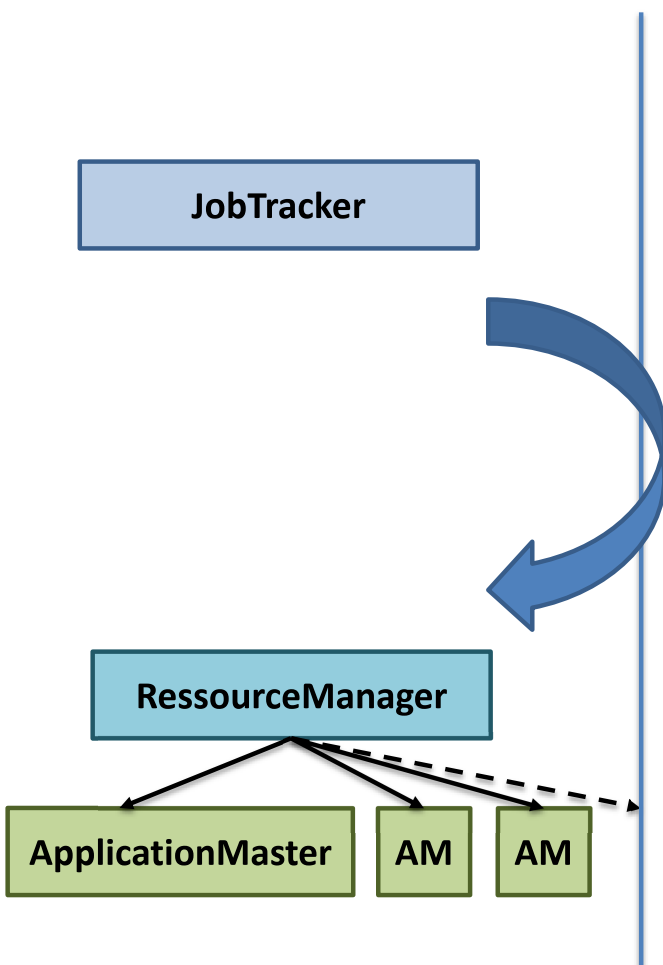
*la capacité d'un équipement à basculer automatiquement vers un réseau ou un système alternatif ou en veille

Re-penser l'architecture du Job Tracker.

- Séparer la gestion des ressources du cluster de la coordination des jobs.
- Utiliser les noeuds esclaves pour gérer les jobs.

ResourceManager et ApplicationMaster.

- ResourceManager remplace le JobTracker et ne gère que les ressources du Cluster.
- Une entité ApplicationMaster est allouée par Application pour gérer les tâches.
- ApplicationMaster est déployée sur les noeuds esclaves.

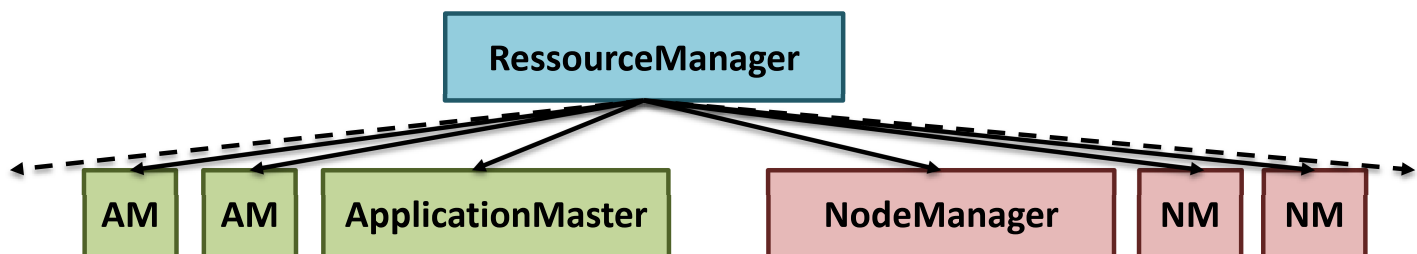


YARN (MapReduce 2) : Architecture (2/7)

- Cette nouvelle version contient aussi un autre composant :

Le NodeManager (NM)

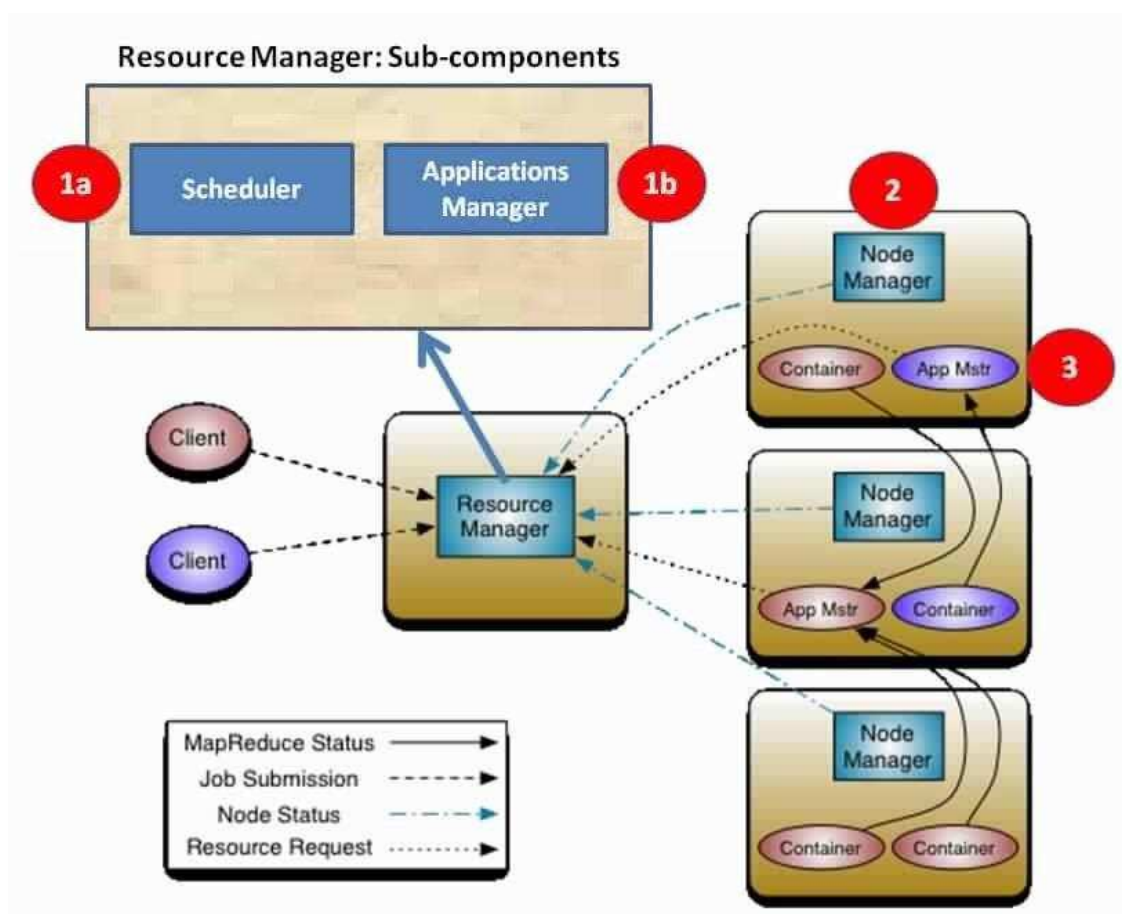
- Permet d'exécuter plus de tâches qui ont du sens pour l'Application Master, pas seulement du Map et du Reduce.
- La taille des ressources est variable (RAM, CPU, network....).



YARN (MapReduce 2) : Architecture (3/7)

- Le **JobTracker** a disparu de l'architecture, ou plus précisément, ses rôles ont été répartis différemment.
- L'architecture est maintenant organisée autour d'un **ResourceManager** dont le périmètre d'action est global au cluster et à des **ApplicationMaster** locaux dont le périmètre est celui d'un job ou d'un groupe de jobs.
- En terme de responsabilités, on peut donc dire que :
JobTracker = ResourceManager + ApplicationMaster.
- La différence, de part le découplage, se trouve dans la multiplicité. En effet, Un **ResourceManager** gère n **ApplicationMaster**, lesquels gèrent chacun n jobs.

YARN (MapReduce 2) : Architecture (4/7)



YARN (MapReduce 2) : Architecture (5/7)

1. ResourceManager

- Le **ResourceManager** est le remplaçant du **JobTracker** du point de vue du **client** qui soumet des jobs (ou plutôt des applications en Hadoop 2) à un cluster Hadoop.
- Il n'a maintenant plus que deux tâches bien distinctes à accomplir :
 - **Scheduler**
 - **ApplicationsManager**

a. Scheduler

- Le Scheduler est responsable de l'allocation des ressources des applications tournant sur le cluster.
- Il s'agit uniquement d'ordonnancement et d'allocation de ressources.
- Les ressources allouées aux applications par le Scheduler pour leur permettre de s'exécuter sont appelées des ***Containers***.

YARN (MapReduce 2) : Architecture (6/7)

❖ Container

- Un Container désigne un regroupement de mémoire, de cpu, d'espace disque, de bande passante réseau, ...

b. ApplicationsManager

- L'ApplicationsManager accepte les soumissions d'applications.
- Une application n'étant pas gérée par le **ResourceManager**, la partie ApplicationsManager ne s'occupe que de négocier le premier **Container** que le **Scheduler** allouera sur un noeud du cluster. La particularité de ce premier **Container** est qu'il contient l'*ApplicationMaster* (diapo suivant).

2. NodeManager

- Les NodeManager sont des agents tournant sur chaque nœud et tenant le **Scheduler** au fait de l'évolution des ressources disponibles. Ce dernier peut ainsi prendre ses décisions d'allocation des **Containers** en prenant en compte des demandes de ressources cpu, disque, réseau, mémoire, ...

YARN (MapReduce 2) : Architecture (7/7)

3. ApplicationMaster

- L'ApplicationMaster est le composant spécifique à chaque application, il est en charge des jobs qui y sont associés.
 - Lancer et au besoin relancer des jobs
 - Négocier les Containers nécessaires auprès du Scheduler
 - Superviser l'état et la progression des jobs.
- Un ApplicationMaster gère donc un ou plusieurs jobs tournant sur un framework donné. Dans le cas de base, c'est donc un ApplicationMaster qui lance un job MapReduce. **De ce point de vue, il remplit un rôle de TaskTracker.**
- ❖ **L'ApplicationsManager est l'autorité qui gère les ApplicationMaster du cluster.** A ce titre, c'est donc via l'ApplicationsManager que l'on peut
 - Superviser l'état des ApplicationMaster
 - Relancer des ApplicationMaster

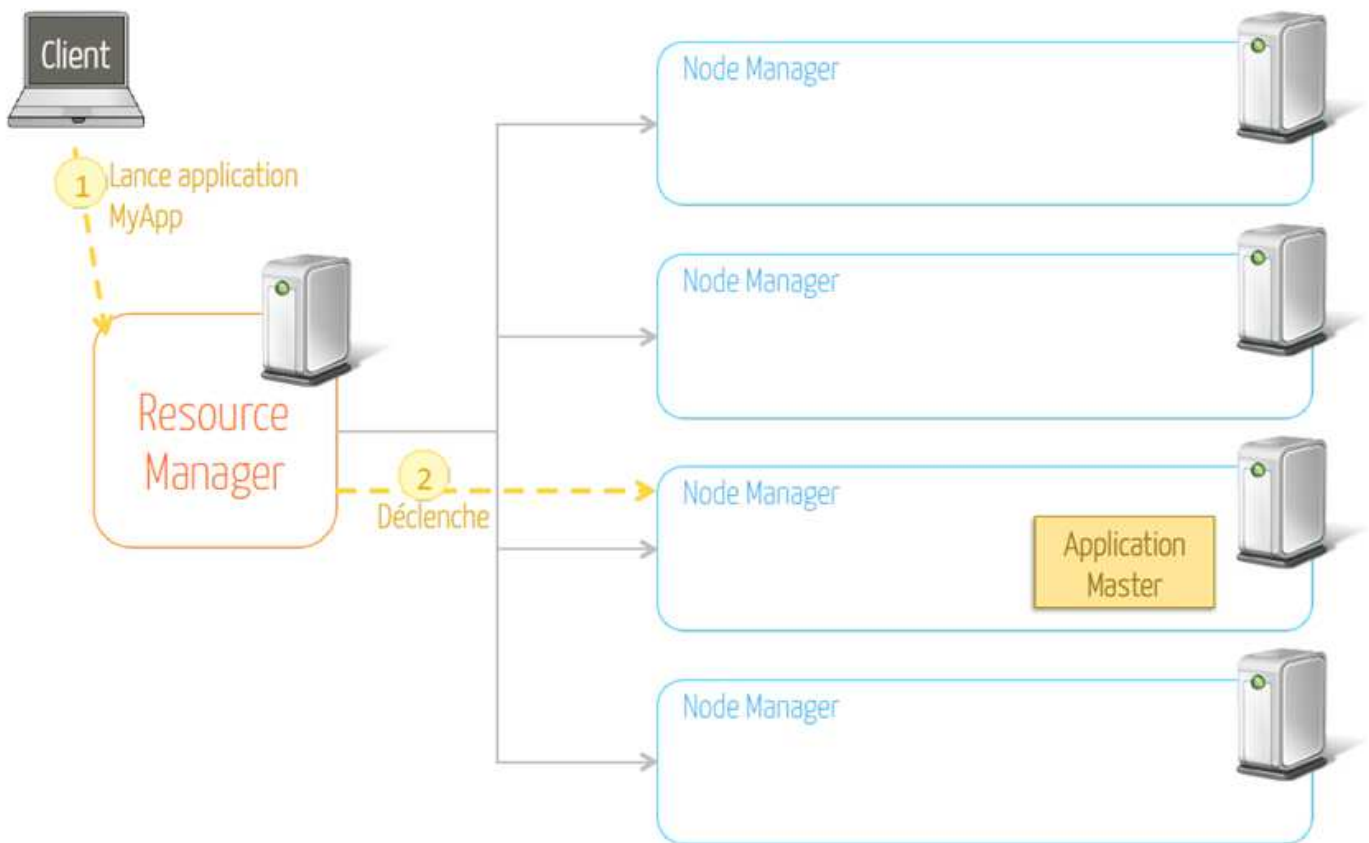
YARN (MapR2): Déroulement de l'exécution

- Le déroulement de l'exécution d'une tâche Hadoop suit les étapes suivantes:
 1. Le client (un outil Hadoop console) va soumettre le travail à effectuer au ResourceManager: une archive java .jar implémentant les opérations Map et Reduce, et également une classe driver (qu'on peut considérer comme le « main » du programme).
 2. Le ResourceManager alloue un container, « Application Master », sur le cluster et y lance la classe « driver » du programme.
 3. Cet « application master » va se lancer et confirmer au ResourceManager qu'il tourne correctement.
 4. Pour chacun des fragments des données d'entrée sur lesquelles travailler, l'Application Master va demander au Resource Manager d'allouer un container en lui indiquant les données sur lesquelles celui-ci va travailler et le code à exécuter.

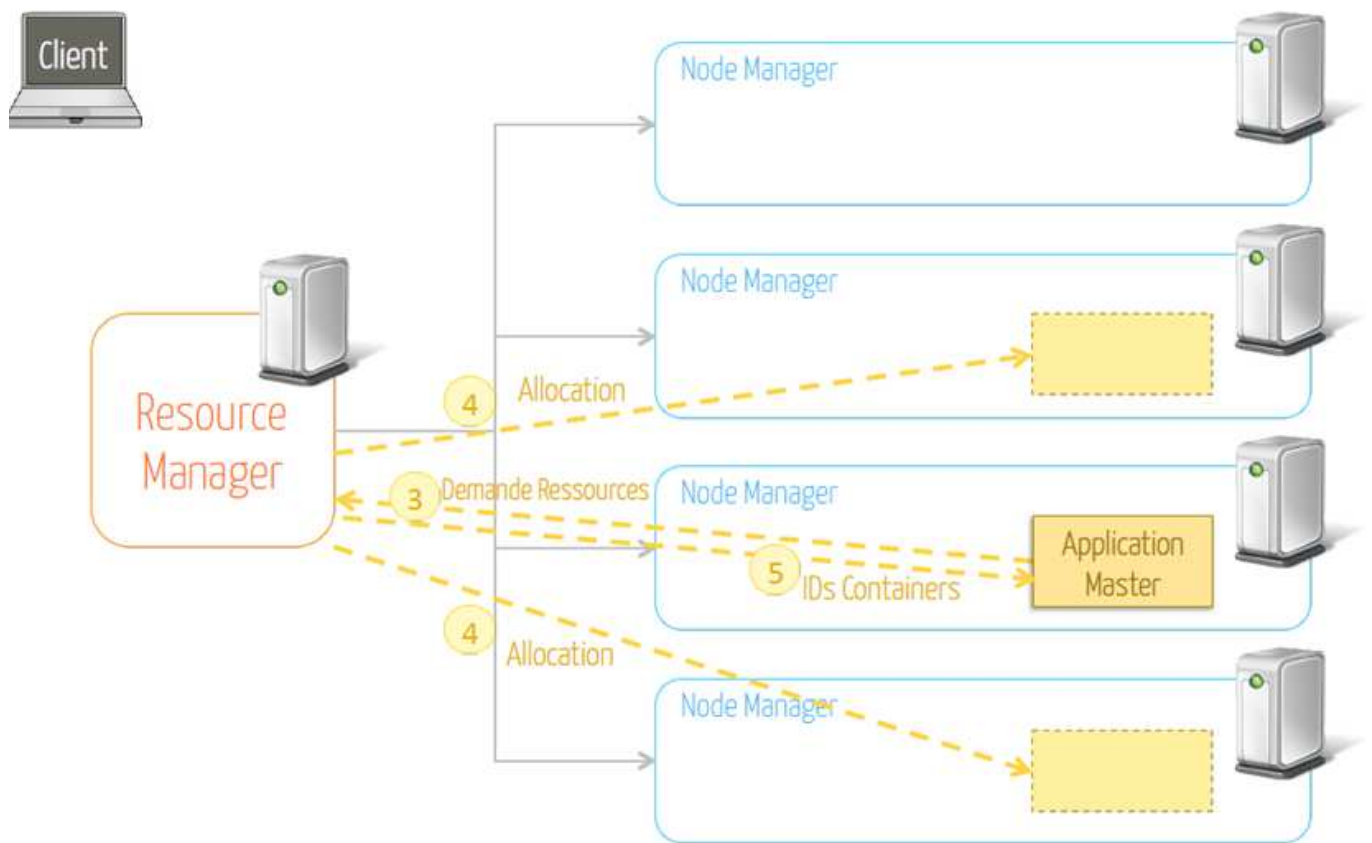
YARN (MapR2): Déroulement de l'exécution

5. L'Application Master va alors lancer le code en question (une classe Java, généralement map ou reduce) sur le container alloué. Il communiquera avec la tâche directement (via un protocole potentiellement propre au programme lancé), sans passer par le ResourceManager.
6. Ces tâches vont régulièrement contacter l'Application Master du programme pour transmettre des informations de progression, de statut, etc. parallèlement, chacun des NodeManager communique en permanence avec le ResourceManager pour lui indiquer son statut en terme de ressources (containers lancés, RAM, CPU), mais sans information spécifique aux tâches exécutées.
7. Pendant l'exécution du programme, le client peut à tout moment contacter le programme en cours d'exécution; pour ce faire, il communique directement avec l'Application Master du programme en contactant le container correspondant sans passer par le ResourceManager du cluster.
8. A l'issue de l'exécution du programme, l'Application Master s'arrête; son container est libéré et est à nouveau disponible pour de futures tâches.

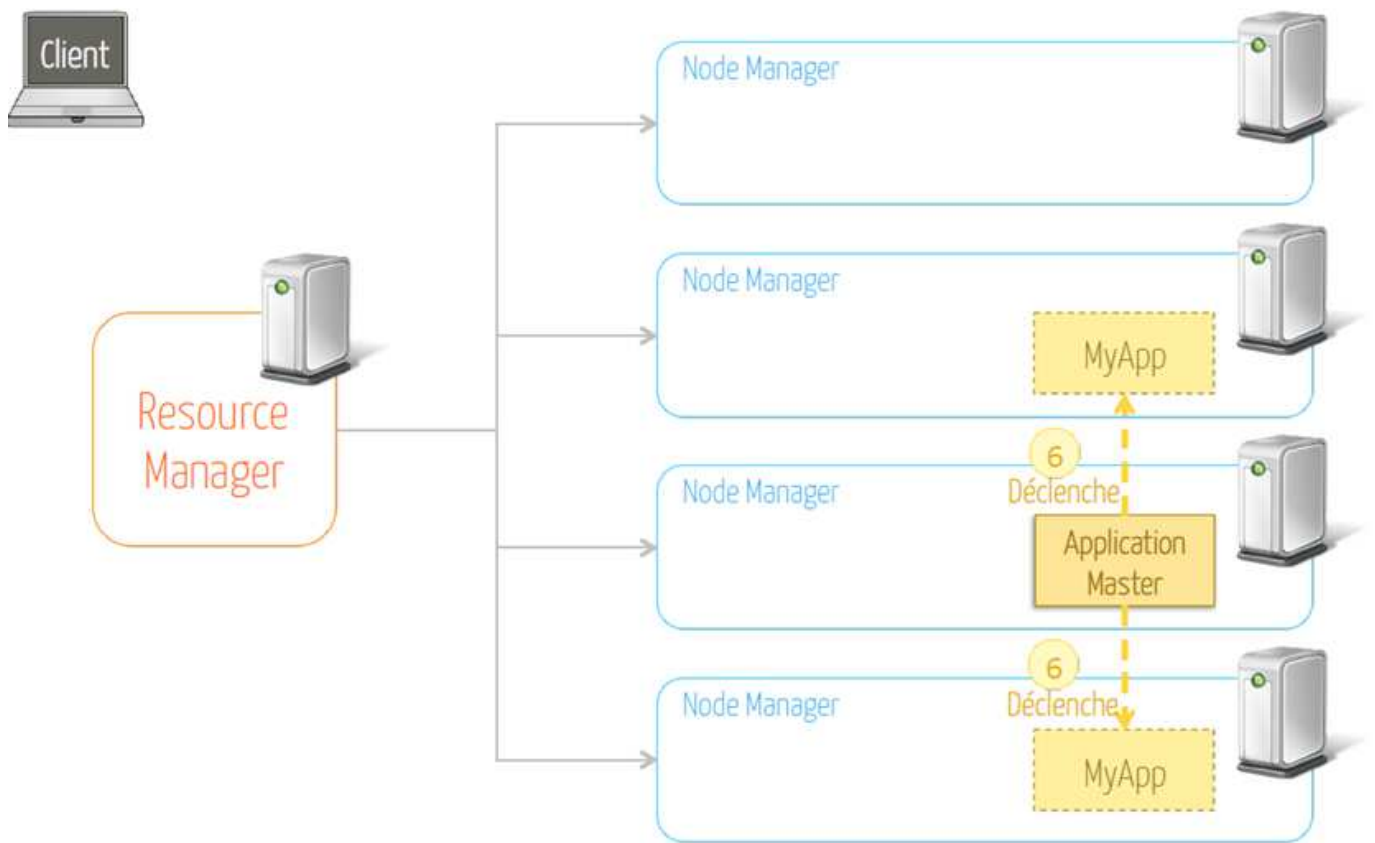
YARN (MapR2) : Lancement d'une Application dans un Cluster Yarn



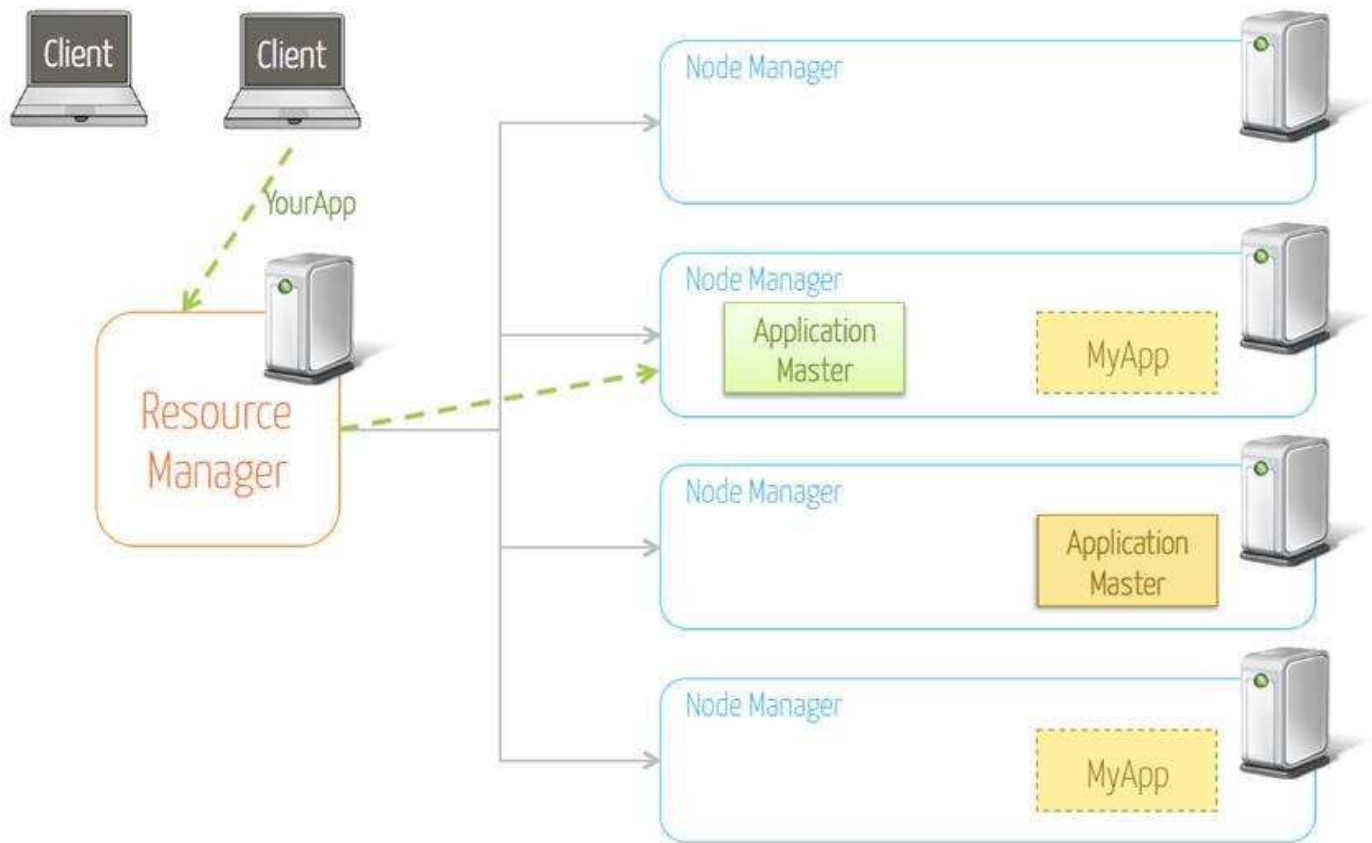
YARN (MapR2) : Lancement d'une Application dans un Cluster Yarn



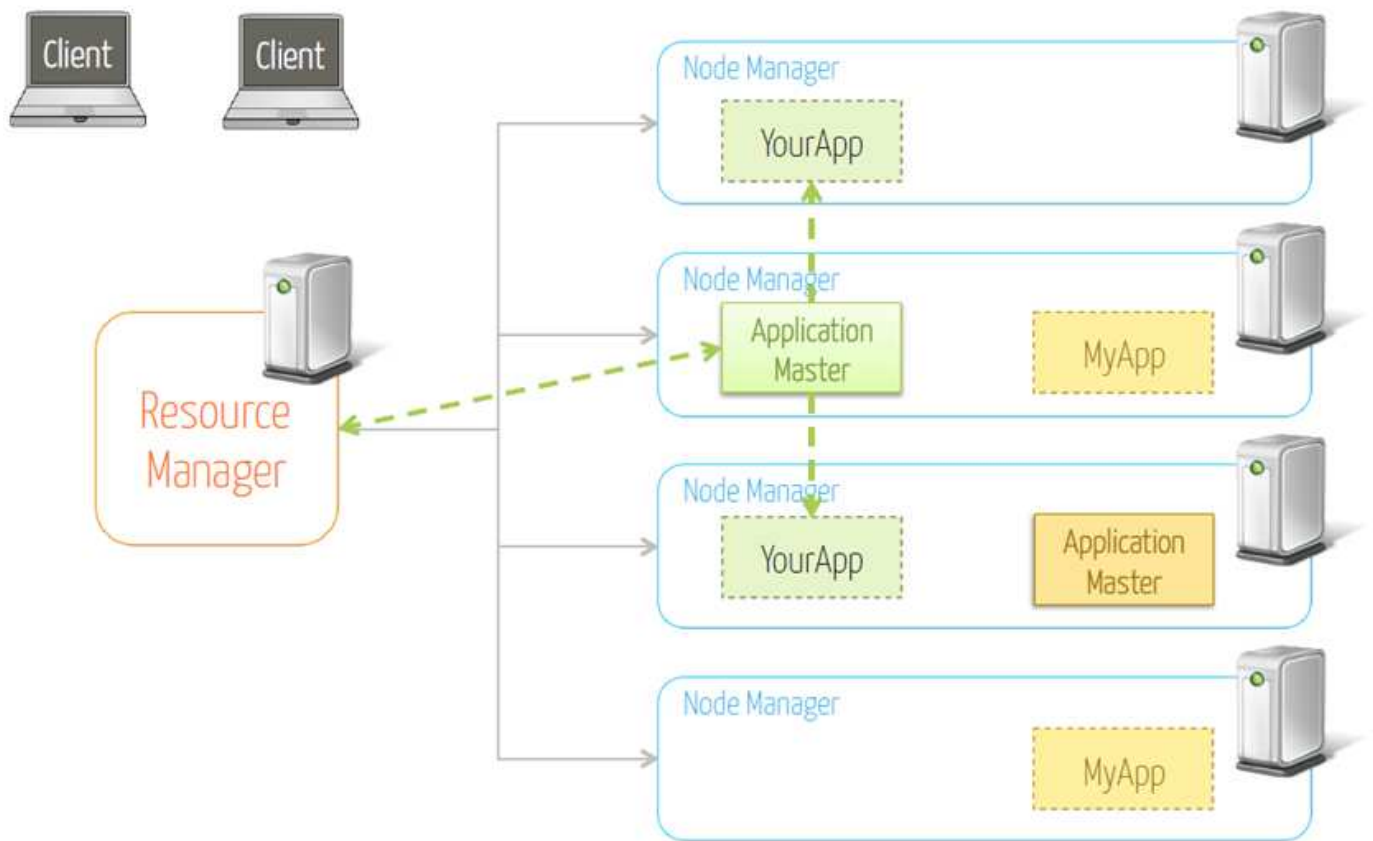
YARN (MapR2) : Lancement d'une Application dans un Cluster Yarn



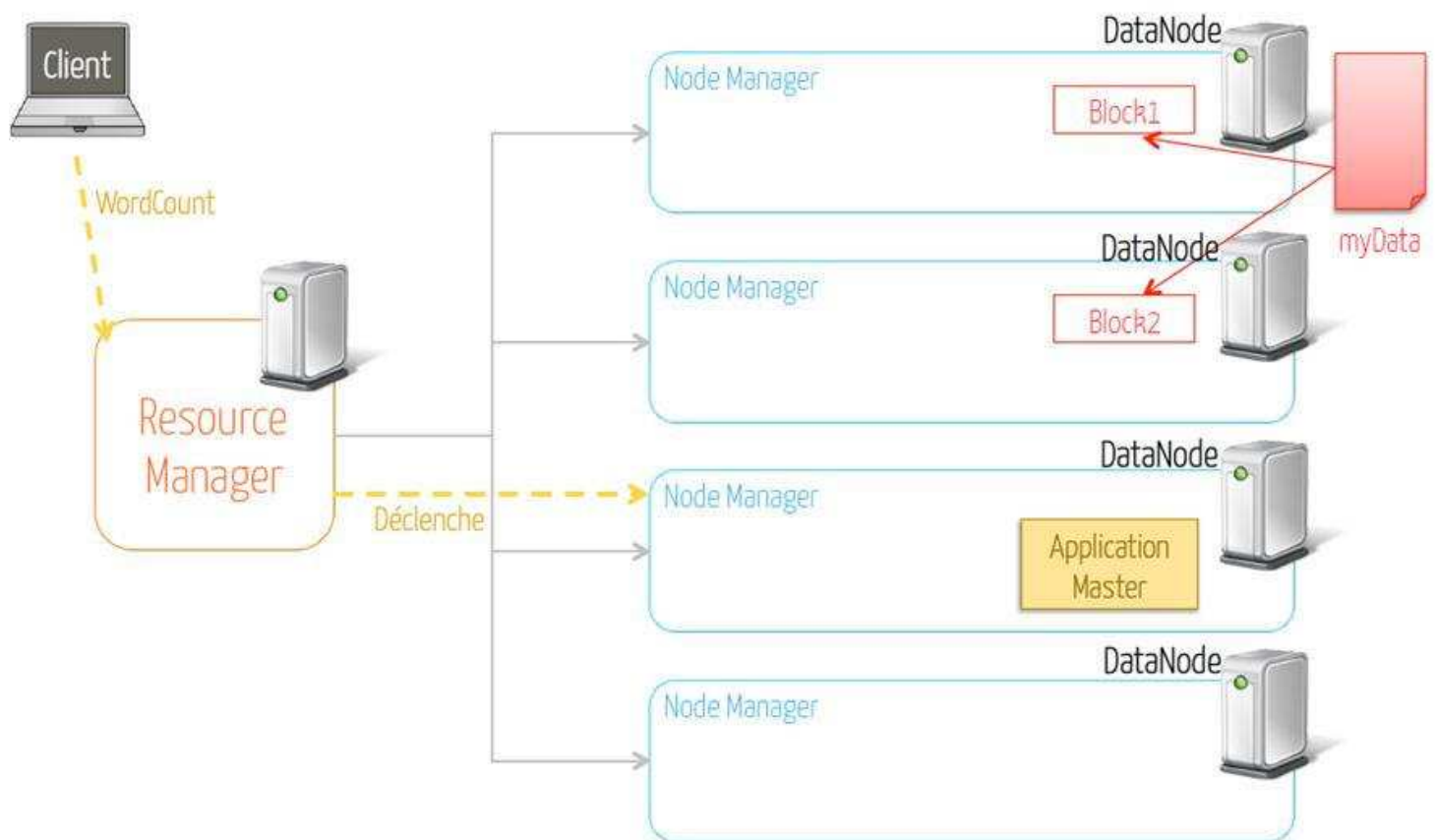
YARN (MapR2) : Lancement d'une Application dans un Cluster Yarn



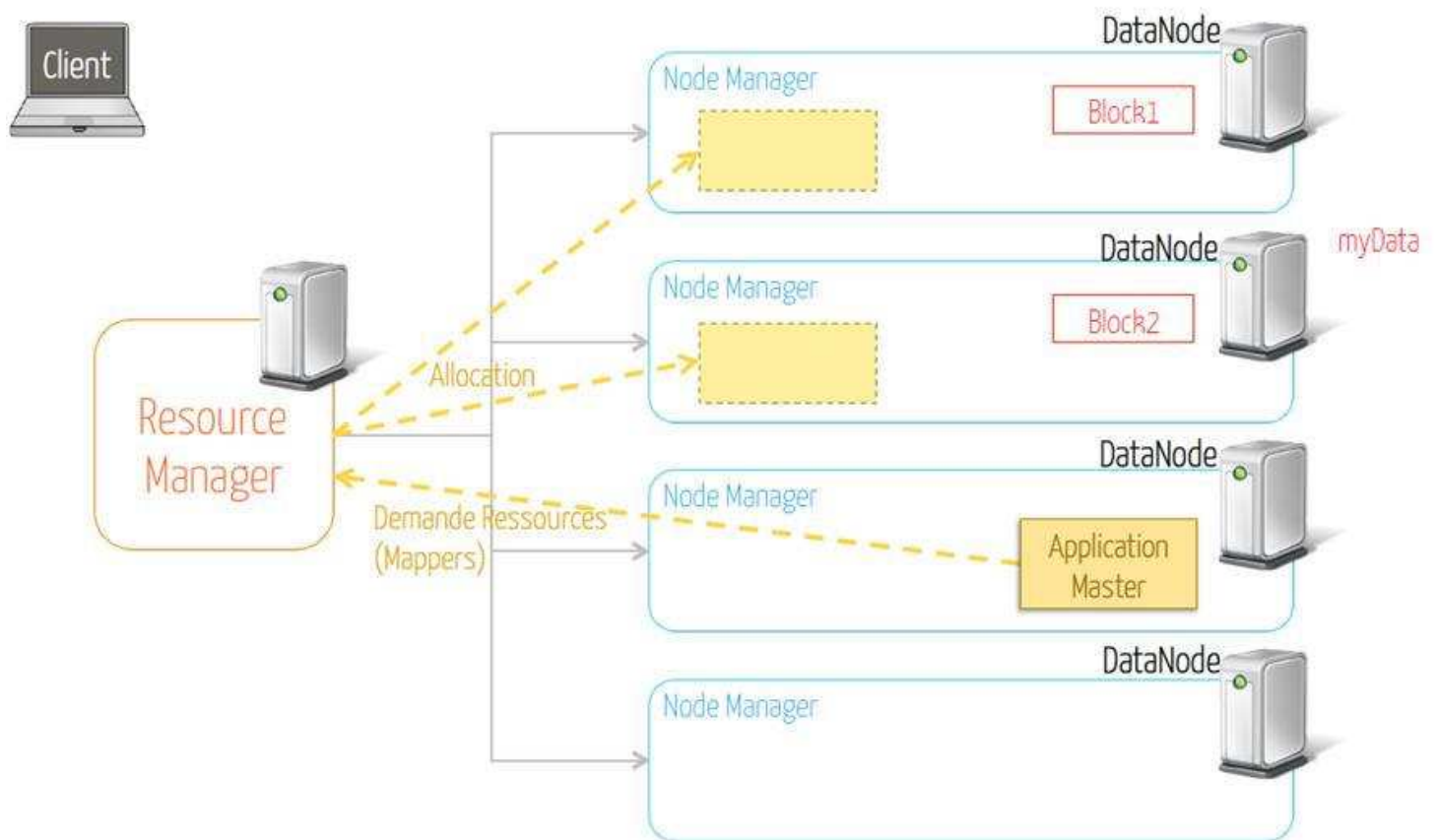
YARN (MapR2) : Lancement d'une Application dans un Cluster Yarn



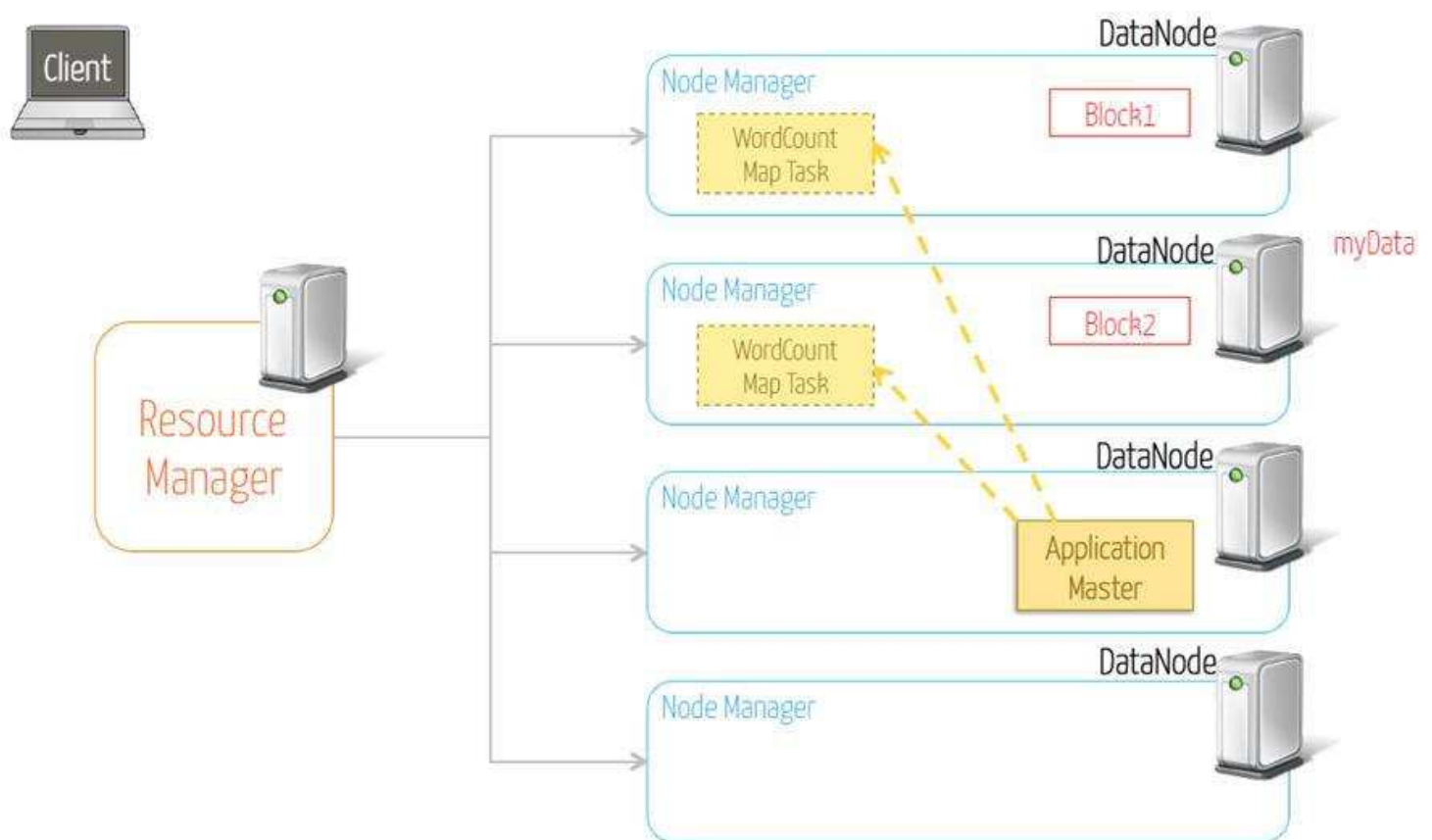
YARN (MapR2) : Exécution d'un Job MR



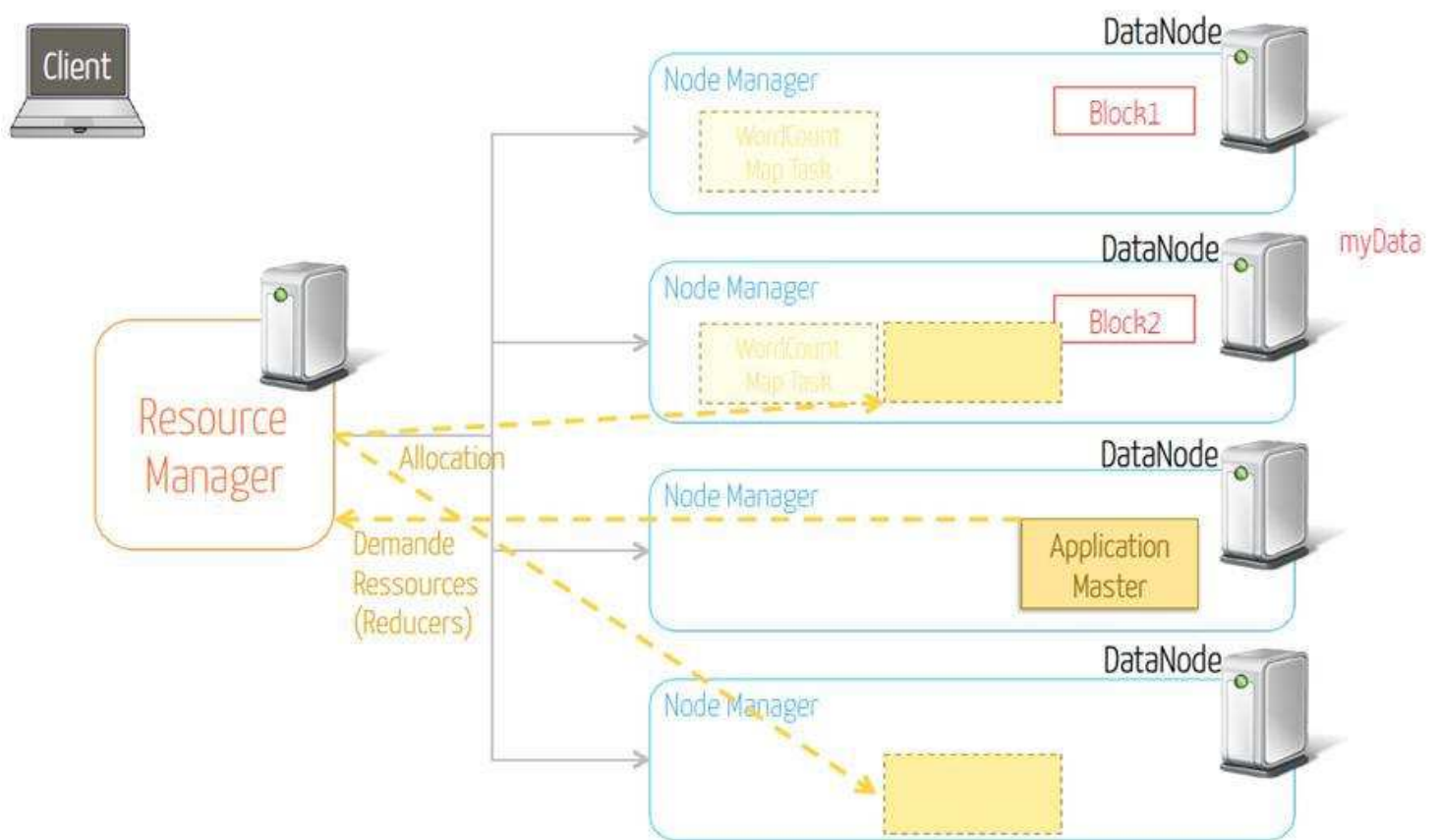
YARN (MapR2) : Exécution d'un Job MR



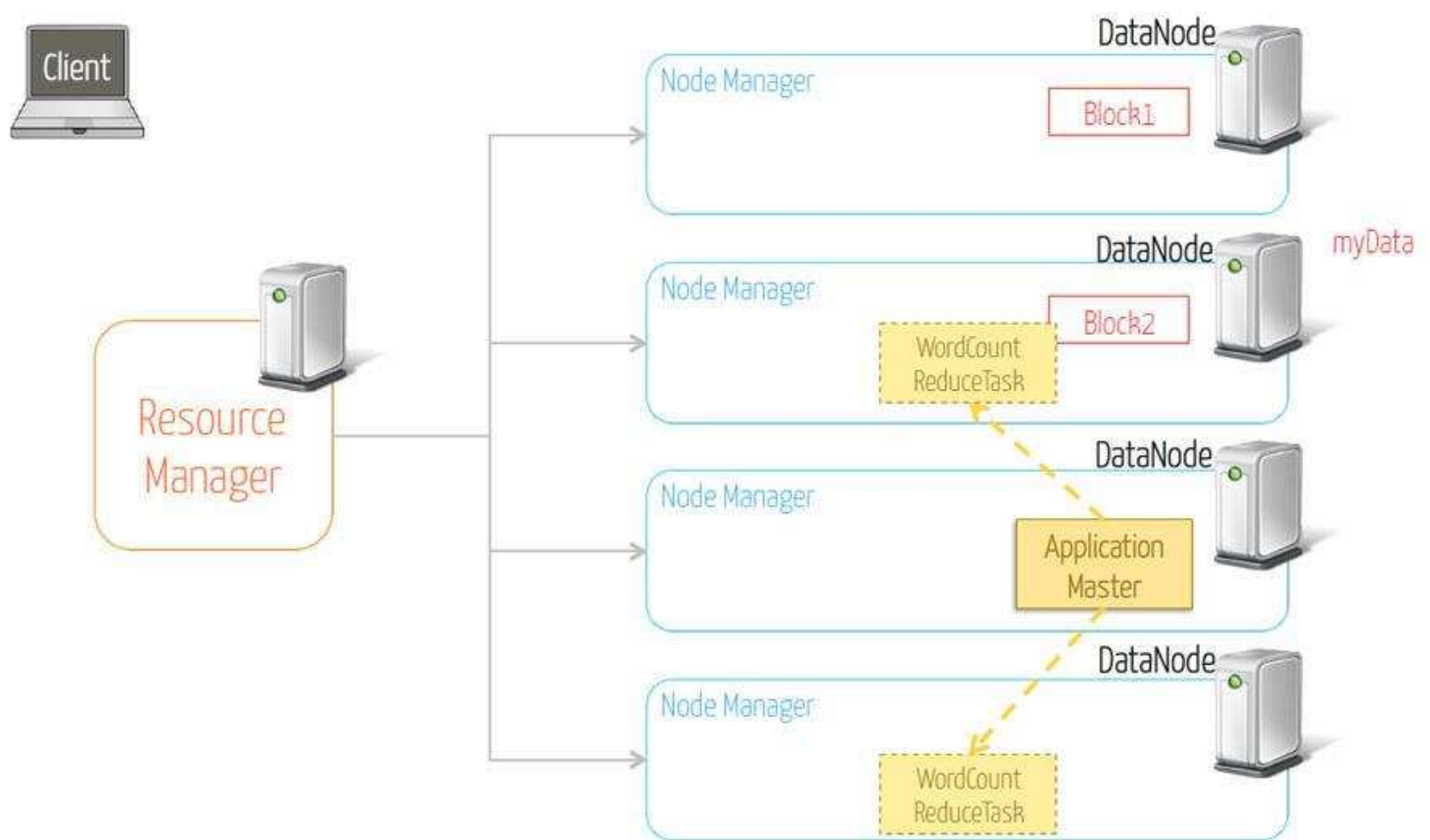
YARN (MapR2) : Exécution d'un Job MR



YARN (MapR2) : Exécution d'un Job MR



YARN (MapR2) : Exécution d'un Job MR



Hadoop 2: HDFS - Remarques

- **De la même façon que l'évolution de MapReduce 2, nous trouvons quelques améliorations de HDFS dans la 2^{ème} version de Hadoop. Ces améliorations ont été déjà expliqué dans la partie consacrée à HDFS.**

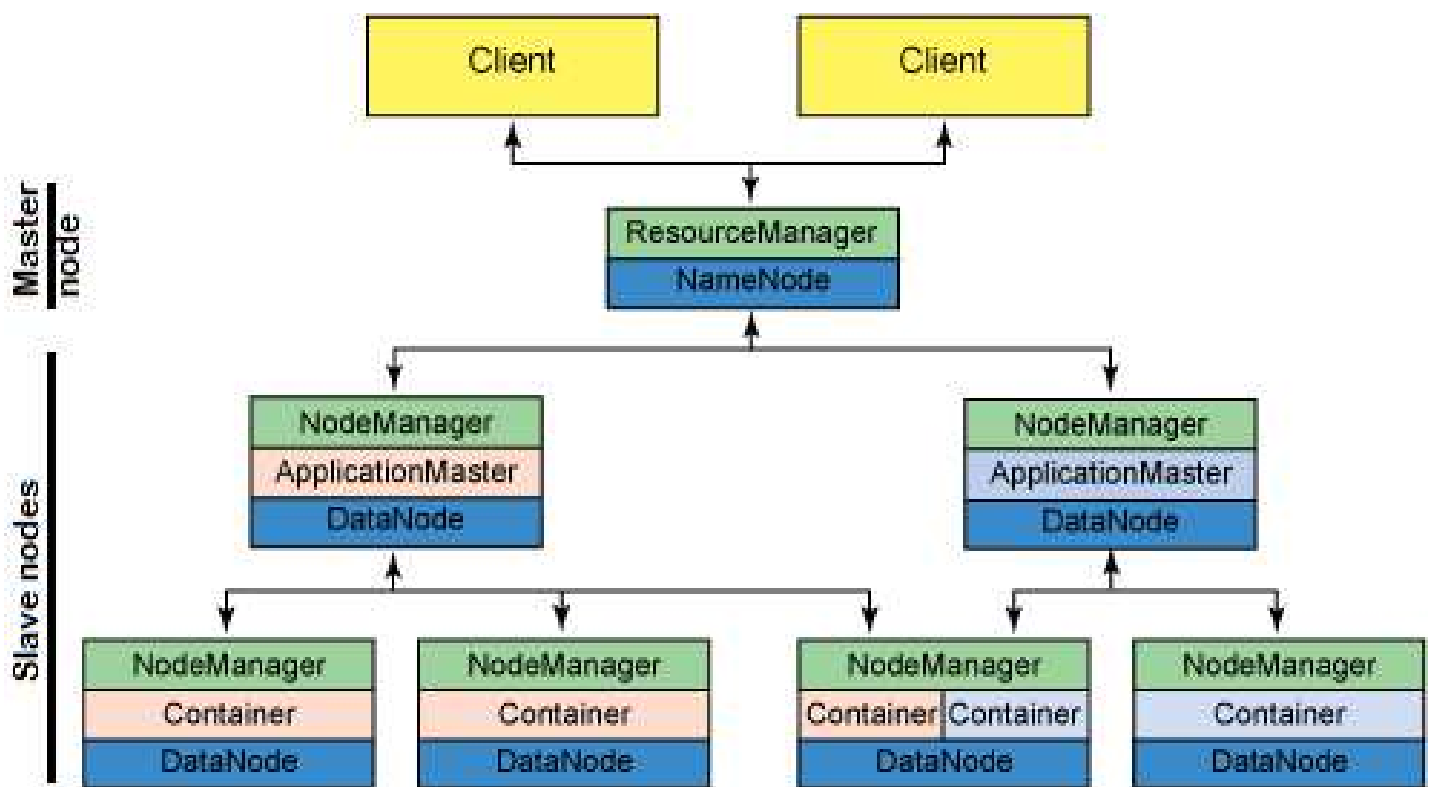
1. Le SPOF du Namenode a disparu

- L'ancienne architecture d'Hadoop imposait l'utilisation d'un seul namenode, composant central contenant les métadonnées du cluster HDFS. Ce namenode était un SPOF (Single Point Of Failure).
- Dans Hadoop 2, on peut mettre deux namenodes en mode actif/attente. Si le Namenode principal est indisponible, le Namenode secondaire prend sa place.

2. La fédération HDFS

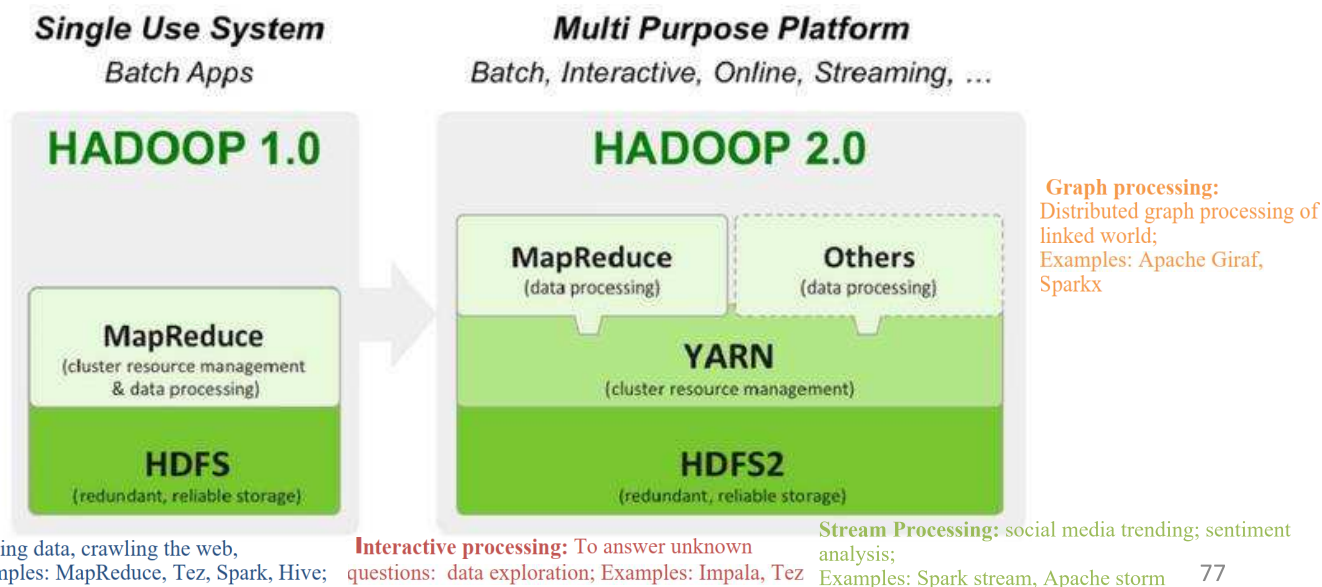
- Dans un cluster HDFS, un namenode correspond à un espace de nommage (namespace). Dans L'ancienne architecture d'Hadoop, on ne pouvait utiliser qu'un namenode par cluster. La fédération HDFS permet de supporter plusieurs namenodes et donc plusieurs namespace sur un même cluster. (Exemple : un NameNode pourrait gérer tous les fichiers sous /user, et un second NameNode peut gérer les fichiers sous /share.)

Hadoop 2: Architecture générale



Hadoop 2: Une évolution architecturale majeure

- Cette remise à plat des rôles a permis aussi de **découpler** Hadoop de MapReduce et, ce faisant, de permettre à des frameworks alternatifs d'être portés directement sur Hadoop et HDFS.
- Cela va donc permettre à Hadoop, outre une meilleure scalabilité, de s'enrichir de nouveaux frameworks couvrant des besoins peu ou pas couverts avec Map Reduce.



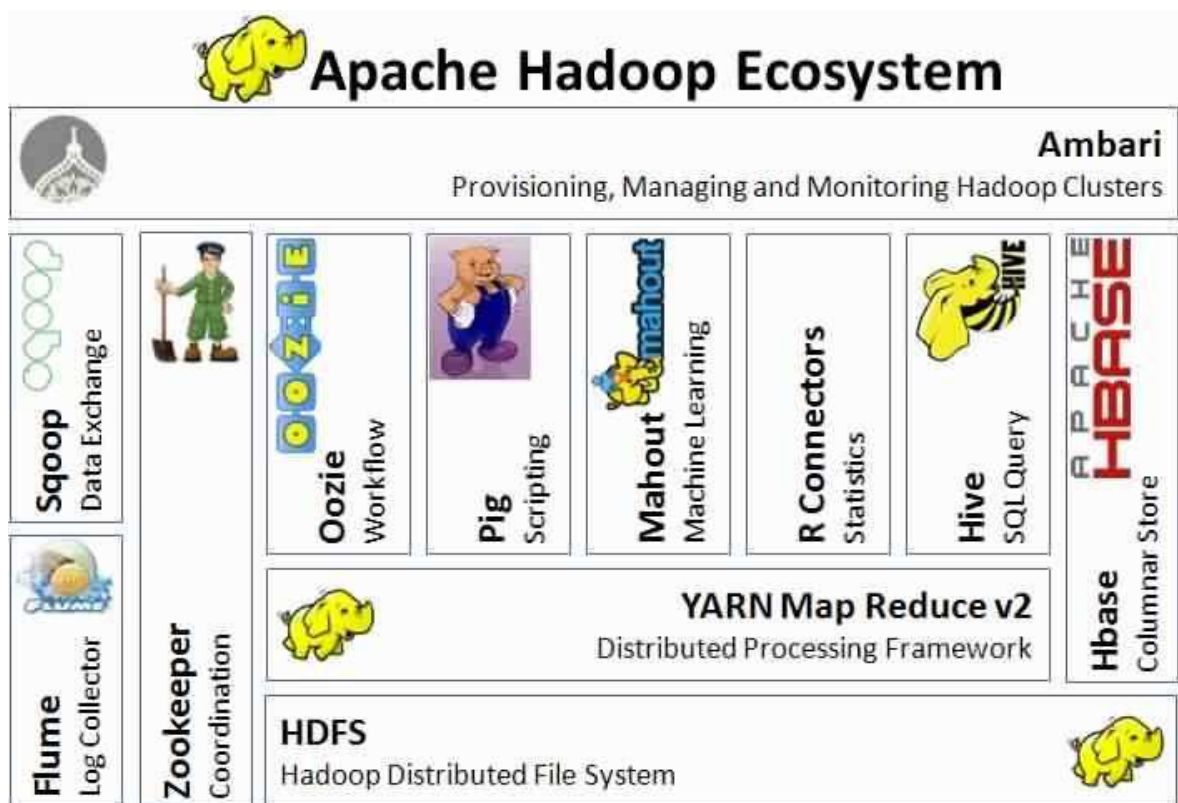
Hadoop 2: Une évolution architecturale majeure

Hadoop se transforme en OS de la donnée !



- Client et cluster peuvent utiliser des versions différentes.
- Des protocoles de communication standardisés et documentés.
- Évolution du framework progressive avec rétro-compatibilité sans destruction des services.

Hadoop 2 : Écosystème (1/2)



Hadoop 2 : Écosystème (2/2)

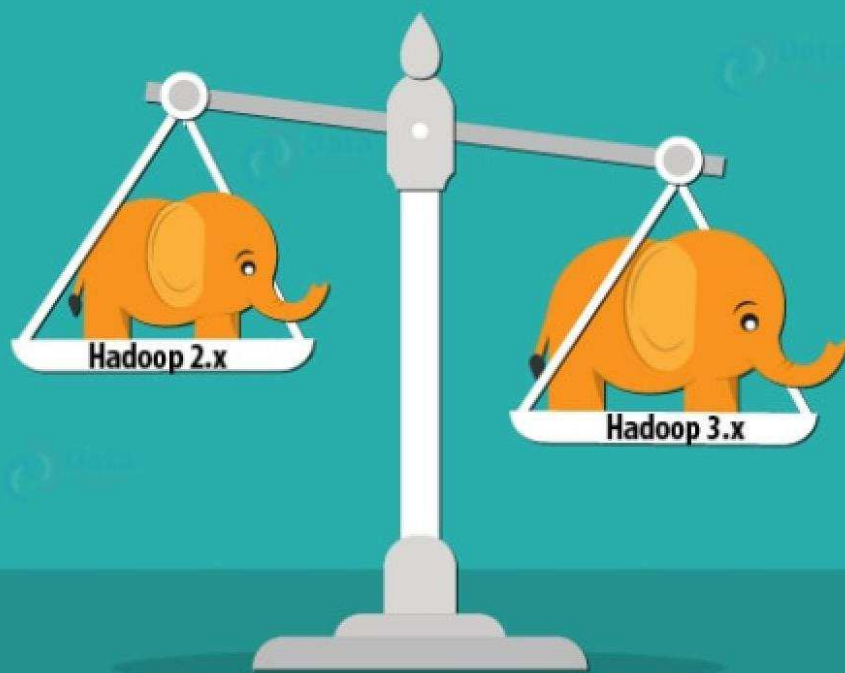
- **Pig** : un langage de haut niveau dédié à l'analyse de gros volumes de données. Il s'adresse aux développeurs habitués à faire des **scripts** via Bash ou Python par exemple.
- **Hive** : un système d'entrepôt de données (Data Warehouse) pour Hadoop qui offre un langage de requête proche de **SQL** pour faciliter les agrégations, le requêtage ad-hoc et l'analyse de gros volumes de données stockés dans des systèmes de fichiers compatibles Hadoop.
- **Hbase** : un système de gestion de base de données **NoSQL** orienté colonnes. Il est distribué, scalable et dédié au big data avec un accès direct et une lecture/écriture temps réel.
- **Sqoop** : un outil conçu pour **transférer** efficacement une masse de données entre Apache Hadoop et un stockage de données **structuré** tel que les bases de données **relationnelles**.
- **Mahout** : un système d'**apprentissage automatique** et d'analyse de données. Il implémente des algorithmes de classification et de regroupement automatique (Machine Learning, DataMining).
- **Flume** : un service distribué, fiable et disponible pour collecter efficacement, agréger et déplacer une grande quantité de **logs**.
- **Zookeeper** : un service centralisé pour maintenir les **configurations**, la **nomenclature**, pour fournir une **synchronisation** distribuée et des services groupés.
- **Oozie** : un système de **flux de travail** (workflow) dont l'objectif est de simplifier la coordination et la séquence de différents traitements (programme Map/Reduce, scripts Pig,...).

Hadoop 2: Conclusion

- **Les changements architecturaux d'Hadoop 2 sont, à minima, intéressants à deux titres :**
- Du point de vue ops, haute disponibilité et la capacité de mutualiser l'infrastructure HDFS.
- Du point de vue dev et métier, le découplage par rapport à MapReduce va permettre de faire émerger d'autres framework. Hadoop n'est donc plus un moteur MapReduce, mais bien un moteur générique pour du calcul distribué.

Hadoop version 3

(1/6)



HADOOP 2.X
VS
HADOOP 3.X

Hadoop version 3 (2/6)

- On présente les différences et les similarités principales entre Hadoop 2.x et Hadoop 3.x.

1. Licence

- Hadoop 2.x – Apache 2.0, Open Source
- Hadoop 3.x – Apache 2.0, Open Source

2. Version minimale supportée de Java

- Hadoop 2.x – Java 7
- Hadoop 3.x – Java 8

3. Tolérance aux pannes

- Hadoop 2.x – La tolérance aux pannes est traitée par la réplication (qui peut être un gaspillage d'espace).
- Hadoop 3.x – La tolérance aux pannes peut être gérée par l'encodage d'effacement « Erasure coding ».

4. Schéma de stockage

- Hadoop 2.x – Utilise un schéma de réplication 3X
- Hadoop 3.x – Supporte l'encodage d'effacement dans HDFS.

Hadoop version 3 (3/6)

5. Frais de stockage (Overhead)

- Hadoop 2.x – HDFS a 200% de frais généraux dans l'espace de stockage.
- Hadoop 3.x – Les frais généraux de stockage ne sont que de 50%.

6. Exemple de surcharge de stockage

- Hadoop 2.x – Si il y a 6 blocs, il y aura 18 blocs occupés dans l'espace en raison du schéma de réplication.
- Hadoop 3.x – S'il y a 6 blocs, il y aura 9 blocs occupés dans l'espace 6 bloc et 3 pour la parité.

7. YARN Timeline Service

- Hadoop 2.x – Utilise un ancien service de chronologie qui a des problèmes d'évolutivité.
- Hadoop 3.x – Améliore le service de de chronologie v2 et améliore l'évolutivité et la fiabilité de ce service.

8. Plage de ports par défaut

- Hadoop 2.x – Certains ports par défaut sont dans la plage de ports éphémère Linux. Ainsi, au moment du démarrage, ils ne se lieront pas.
- Hadoop 3.x – Ces ports ont été déplacés hors de la plage de ports éphémère.

Hadoop version 3 (4/6)

10. Outils

- Hadoop 2.x – Utilise Hive, Pig, Tez, Hama, Giraph et d'autres outils Hadoop.
- Hadoop 3.x – Utilise Hive, Pig, Tez, Hama, Giraph et d'autres outils Hadoop.

11. Système de fichiers compatible

- Hadoop 2.x – HDFS (FS par défaut), système de fichiers FTP: stocke toutes ses données sur les serveurs FTP accessibles à distance. Système de fichiers Amazon S3 (Service de stockage simple) Système de fichiers Windows Azure Storage Blobs (WASB).
- Hadoop 3.x – Il prend en charge tous les systèmes de fichiers précédents, ainsi que Microsoft Azure Data Lake.

12. Compatibilité de l'API MapReduce

- Hadoop 2.x – L'API MR est compatible avec les programmes Hadoop 1.x pour s'exécuter sur Hadoop 2.X
- Hadoop 3.x – Ici aussi, l'API MR est compatible avec les programmes Hadoop 1.x pour s'exécuter sur Hadoop 2.X.

Hadoop version 3 (5/6)

13. Slots/Container

- Hadoop 2.x – Hadoop 1.x fonctionne avec le concept de slots, mais Hadoop 2.x fonctionne avec le concept du conteneur. Dans le conteneur, nous pouvons exécuter la tâche générique.
- Hadoop 3.x – Il fonctionne également sur le concept d'un conteneur.

14. Point unique de défaillance

- Hadoop 2.x – Possède des fonctionnalités pour surmonter SPOF, chaque fois que Namenode échoue, il se rétablit automatiquement.
- Hadoop 3.x – De même, il se rétablit automatiquement sans intervention manuelle pour surmonter le SPOF du Namenode.

15. Fédération HDFS

- Hadoop 2.x – Dans Hadoop 1.0, il existe un seul Namenode pour gérer tout l'espace de noms, mais dans Hadoop 2.0 de multiples Namenodes pour plusieurs espaces de noms.
- Hadoop 3.x – Dans Hadoop 3.x , il existe également plusieurs Namenodes pour plusieurs espaces de noms.

Hadoop version 3

(6/6)

16. Évolutivité

- Hadoop 2.x – Nous pouvons étendre jusqu'à 10 000 noeuds par cluster.
- Hadoop 3.x – Meilleure évolutivité. Etendre plus de 10 000 noeuds par cluster.

17. HDFS Snapshot

- Hadoop 2.x – Hadoop 2 ajoute le support du Snapshot. Il fournit une reprise après un désastre et une protection contre les erreurs de l'utilisateur.
- Hadoop 3.x – Hadoop 3 supporte également la fonctionnalité de snapshot.

18. Cluster Resource Management

- Hadoop 2.x – Utilise YARN. Il améliore l'évolutivité et la haute disponibilité.
- Hadoop 3.x – De même, il utilise YARN avec toutes ses fonctionnalités.

19. Plate-forme

- Hadoop 2.x – Peut servir en tant que plate-forme pour une large variété d'analyses de données possibles pour exécuter le traitement d'événements, le streaming et les opérations temps réel.
- Hadoop 3.x – Ici, il est également possible d'exécuter ces traitements puisque ça sera de même au-dessus de YARN.

Sources

- Cours
 - Ce chapitre est pris essentiellement du cours : Hadoop / Big Data - MBDS - Benjamin Renault (avec quelques modifications).
- Articles
 - « Hadoop et le "Big Data », La Technologie Hadoop au coeur des projets "Big Data" - Stéphane Goumard – Tech day Hadoop, Spark - Arrow-Institute.
 - Comparison Between Hadoop 2.x vs Hadoop 3.x – Data Flair.